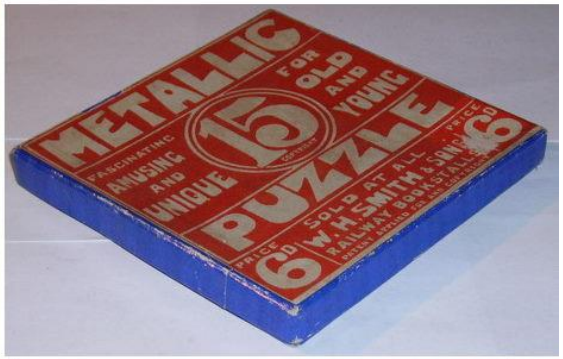
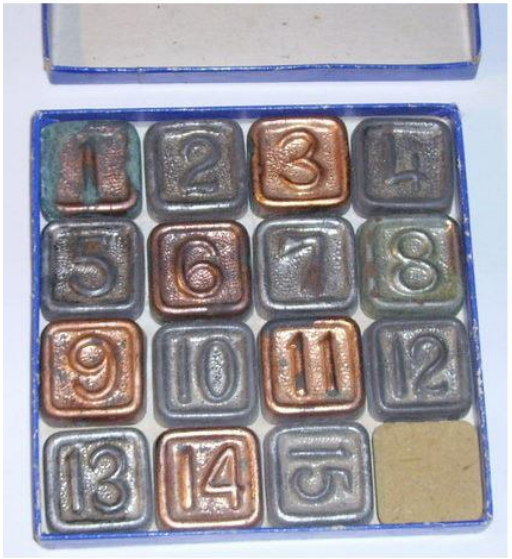
Artificial Intelligence (Trí tuệ nhân tạo)
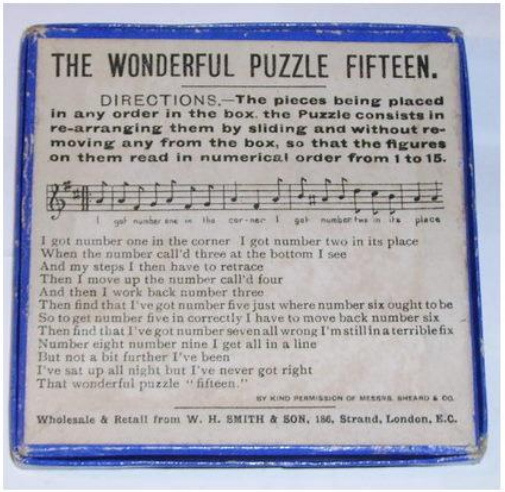
Solving Problems by searching

(Giải quyết vấn đề bằng tìm kiếm)









THE WONDERFUL PUZZLE FIFTEEN.

DIRECTIONS.—**The pieces being placed in any order in the box, the Puzzle consists in re-arranging them by sliding and without removing any from the box, so that the figures on them read in numerical order from 1 to 15.**

I got number one in the corner I got number two in its place
When the number call'd three at the bottom I see
And my steps I then have to retrace
Then I move up the number call'd four
And then I work back number three
Then find that I've got number five just where number six ought to be
So to get number five in correctly I have to move back number six
Then find that I've got number seven all wrong I'm still in a terrible fix
Number eight number nine I get all in a line
But not a bit further I've been
I've sat up all night but I've never got right
That wonderful puzzle "fifteen."

BY KIND PERMISSION OF MESSRS. SHEARD & CO.

Wholesale & Retail from W. H. SMITH & SON, 186, Strand, London, E.C.

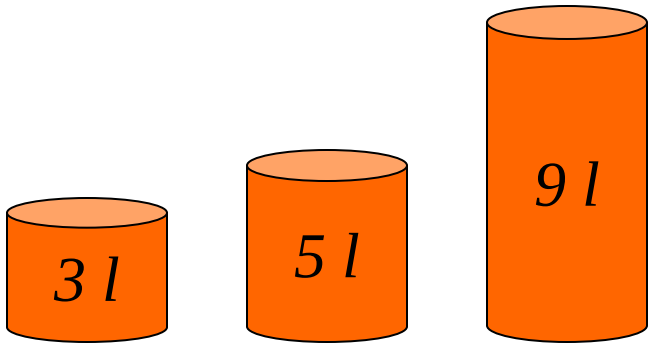
Giải quyết vấn đề bằng tìm kiếm

- **Giới thiệu về phương pháp giải quyết vấn đề bằng tìm kiếm**

- **Các phương pháp tìm kiếm mù**
  - Phát biểu bài toán
  - Các chiến lược tìm kiếm: Tìm kiếm theo chiều sâu trước (depth-first), tìm kiếm theo chiều rộng trước (breadth-first)

- **Các phương pháp tìm kiếm theo kinh nghiệm**
  - Các chiến lược tìm kiếm: Tốt nhất đầu tiên (best-first), A*
  - Hàm kinh nghiệm (Heuristic functions)

Ví dụ: Bài toán đong nước

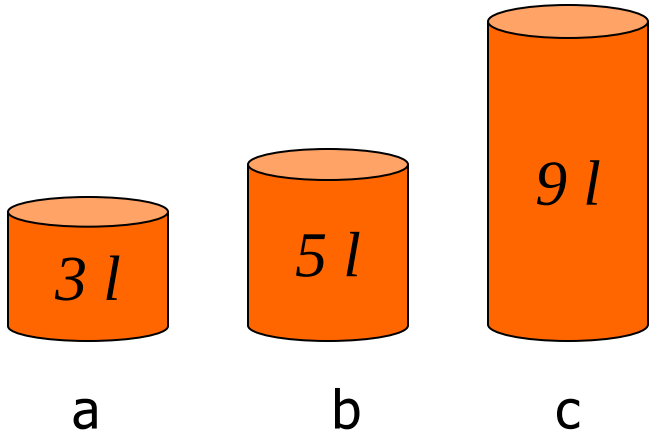


Phát biểu bài toán: Sử dụng 3 bình nước với dung tích 3 lít, 5 lít, và 9 lít, hãy tìm cách đong được 7 lit nước

Ví dụ: Bài toán đong nước

- **Giải pháp 1**

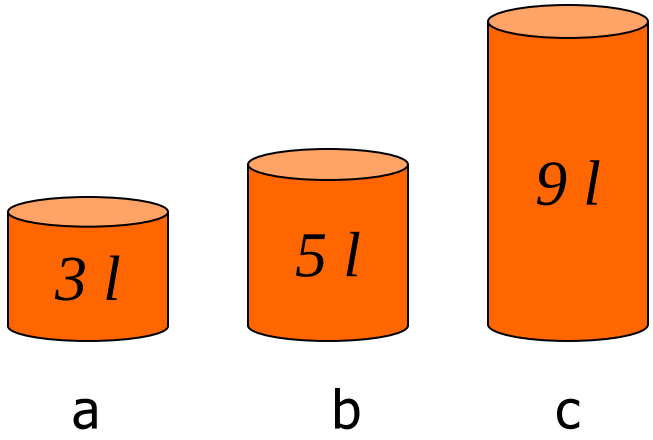
a	b	c	
0	0	0	start



Ví dụ: Bài toán đong nước

- **Giải pháp 1**

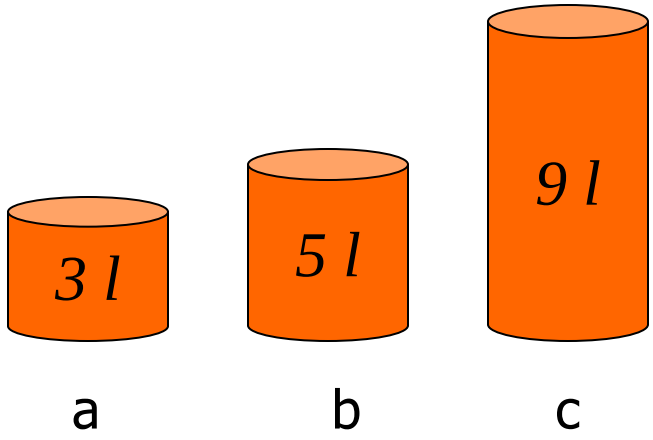
a	b	c	
0	0	0	start
3	0	0	



Ví dụ: Bài toán đong nước

- **Giải pháp 1**

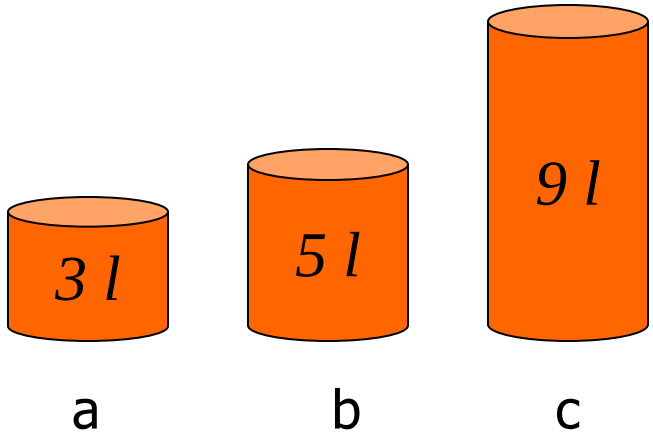
a	b	c	
0	0	0	start
3	0	0	
0	0	3	



Ví dụ: Bài toán đong nước

- **Giải pháp 1**

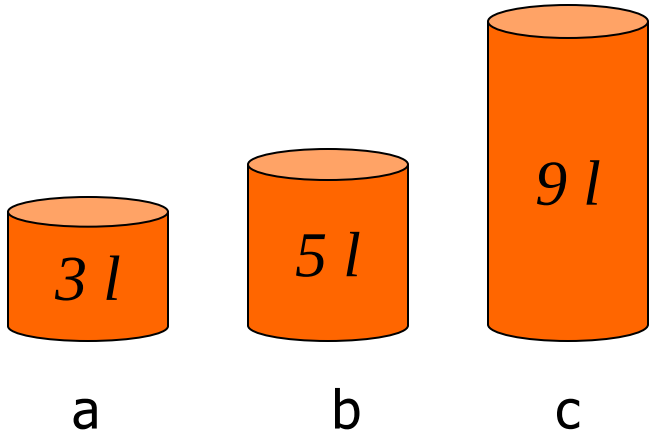
a	b	c	
0	0	0	start
3	0	0	
0	0	3	
3	0	3	



Ví dụ: Bài toán đong nước

- **Giải pháp 1**

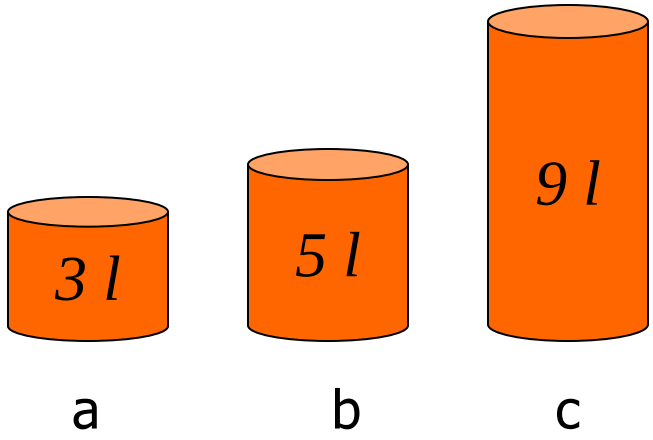
a	b	c	
0	0	0	start
3	0	0	
0	0	3	
3	0	3	
0	0	6	



Ví dụ: Bài toán đong nước

- **Giải pháp 1**

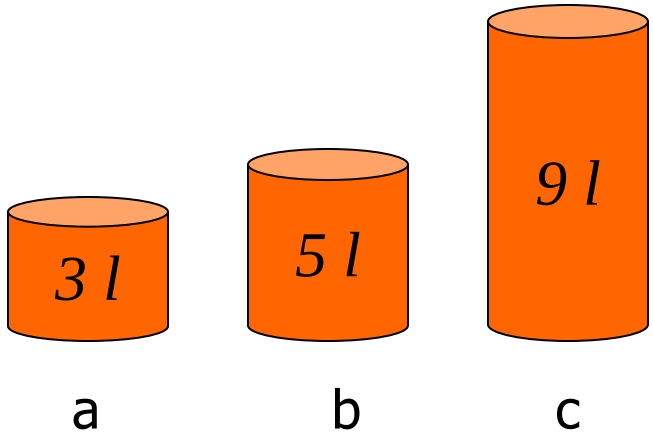
a	b	c	
0	0	0	start
3	0	0	
0	0	3	
3	0	3	
0	0	6	
3	0	6	



Ví dụ: Bài toán đong nước

- **Giải pháp 1**

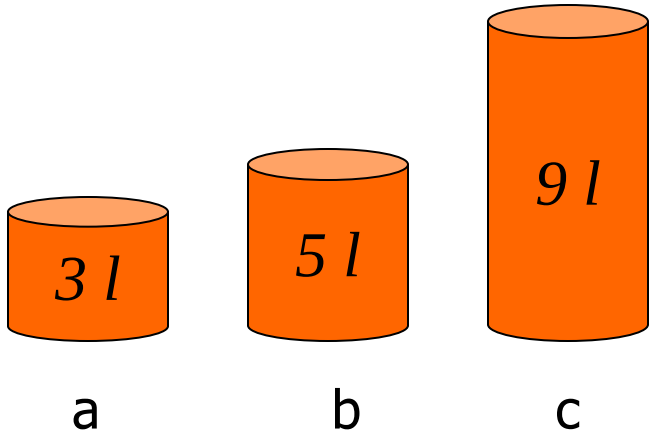
a	b	c	
0	0	0	start
3	0	0	
0	0	3	
3	0	3	
0	0	6	
3	0	6	
0	3	6	



Ví dụ: Bài toán đong nước

- **Giải pháp 1**

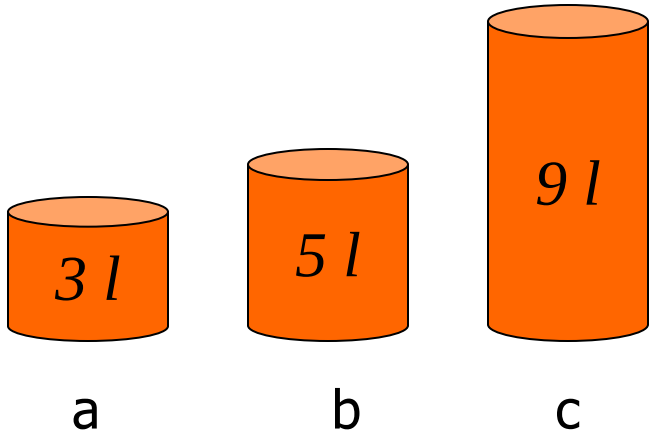
a	b	c	
0	0	0	start
3	0	0	
0	0	3	
3	0	3	
0	0	6	
3	0	6	
0	3	6	
3	3	6	



Ví dụ: Bài toán đong nước

- **Giải pháp 1**

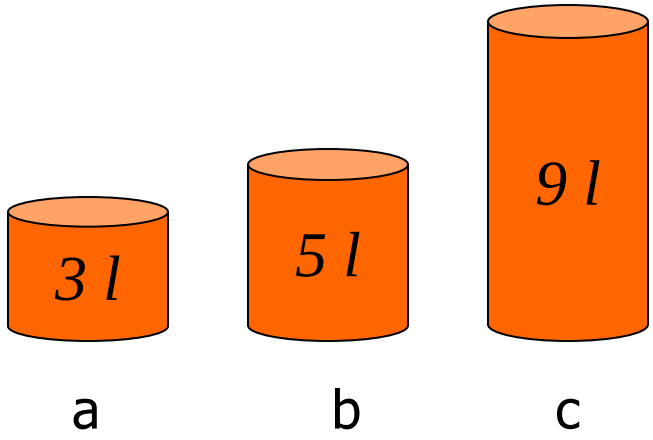
a	b	c	
0	0	0	start
3	0	0	
0	0	3	
3	0	3	
0	0	6	
3	0	6	
0	3	6	
3	3	6	
1	5	6	



Ví dụ: Bài toán đong nước

- **Giải pháp 1**

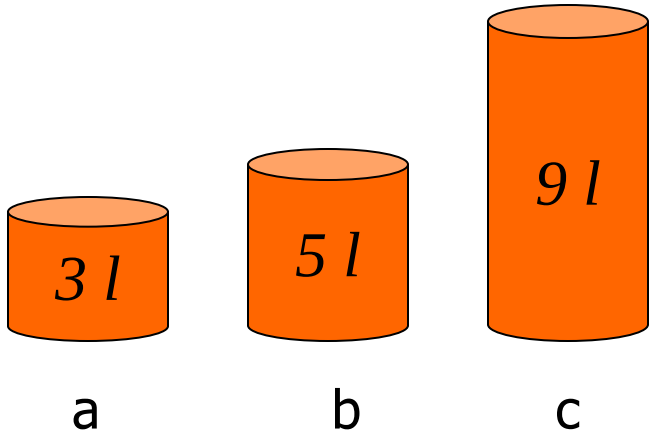
a	b	c	
0	0	0	start
3	0	0	
0	0	3	
3	0	3	
0	0	6	
3	0	6	
0	3	6	
3	3	6	
1	5	6	
0	5	**7**	**goal**



Ví dụ: Bài toán đong nước

- **Giải pháp 2**

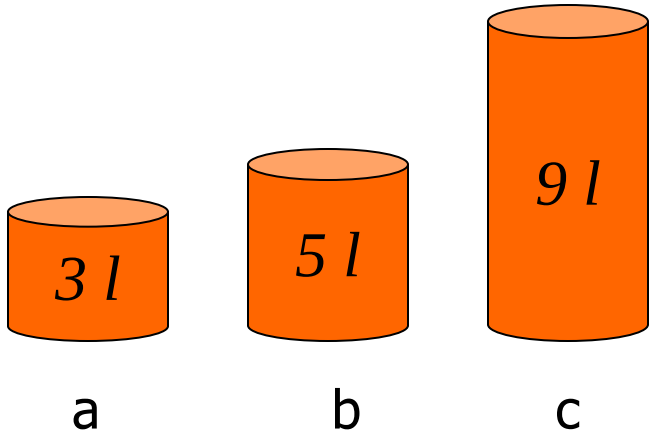
a	b	c	
0	0	0	start
0	5	0	



Ví dụ: Bài toán đong nước

- **Giải pháp 2**

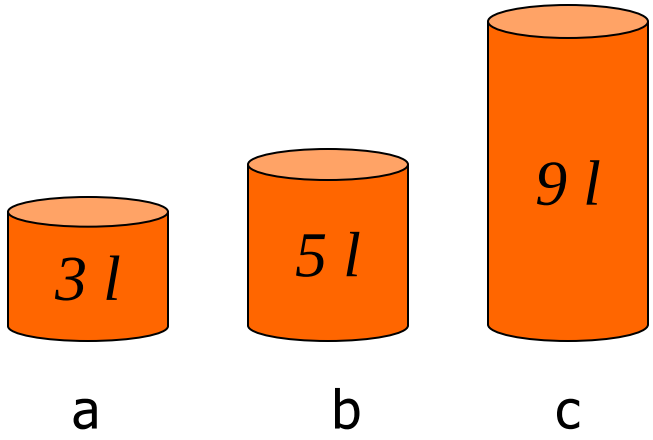
a	b	c	
0	0	0	start
0	5	0	
3	2	0	



Ví dụ: Bài toán đong nước

- **Giải pháp 2**

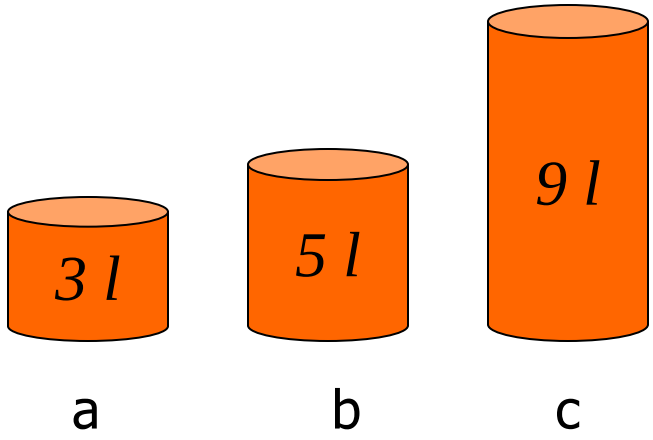
a	b	c	
0	0	0	start
0	5	0	
3	2	0	
3	0	2	



Ví dụ: Bài toán đong nước

- **Giải pháp 2**

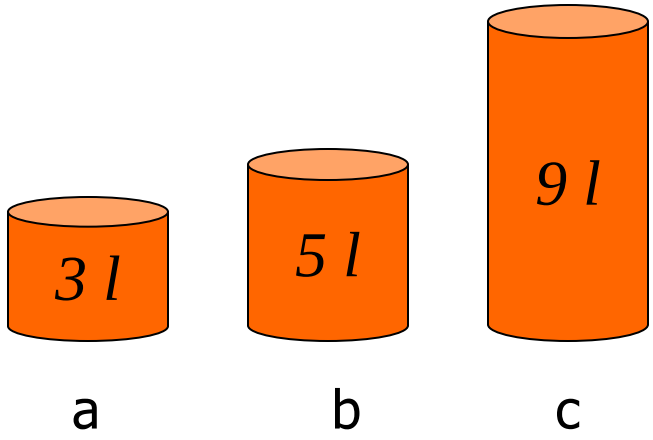
a	b	c	
0	0	0	start
0	5	0	
3	2	0	
3	0	2	
3	5	2	



Ví dụ: Bài toán đong nước

- **Giải pháp 2**

a	b	c	
0	0	0	start
0	5	0	
3	2	0	
3	0	2	
3	5	2	
3	**0**	**7**	**goal**



Giải pháp nào tốt hơn?

- **Giải pháp 1:**

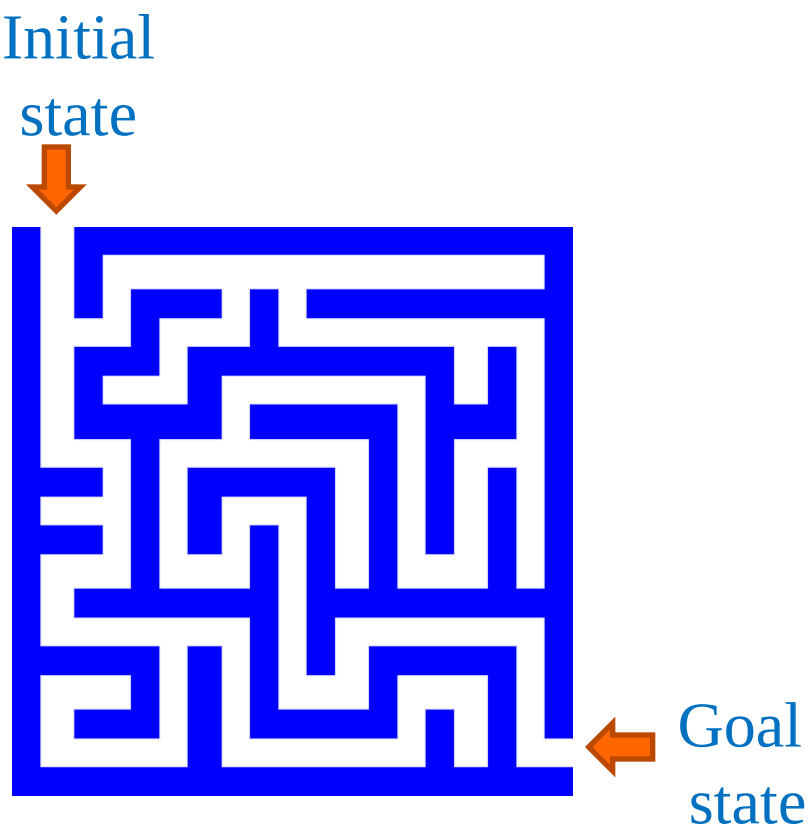
a	b	c	
0	0	0	start
3	0	0	
0	0	3	
3	0	3	
0	0	6	
3	0	6	
0	3	6	
3	3	6	
1	5	6	
0	5	**7**	**goal**

- **Giải pháp 2:**

a	b	c	
0	0	0	start
0	5	0	
3	2	0	
3	0	2	
3	5	2	
3	**0**	**7**	**goal**

Các thành phần của bài toán tìm kiếm

- **Initial state**
- **Actions**
- **Transition model**
  - What state results from performing a given action in a given state?
- **Goal state**
- **Path cost**
  - Assume that it is a sum of nonnegative *step costs*



- The **optimal solution** is the sequence of actions that gives the *lowest* path cost for reaching the goal

Ví dụ: Romania

- On vacation in Romania; currently in Arad
- Flight leaves tomorrow for Bucharest



- **Initial state**
  - Arad

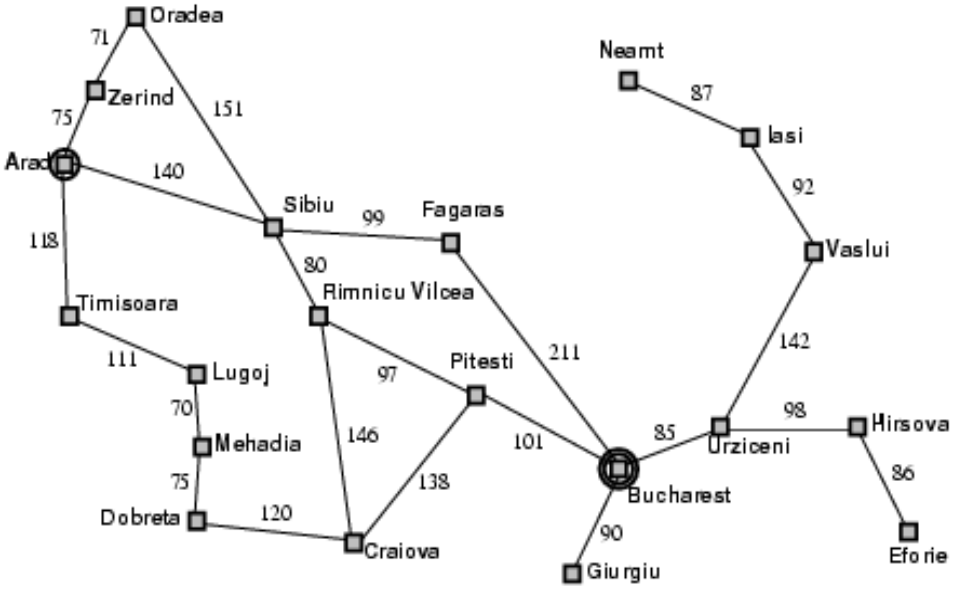
  ~ In(Arad)
- **Actions**
  - Go from one city to another

  ~ Go(Sibiu), Go(Timisoara), Go(Zerind)
- **Transition model**
  - If you go from city A to city B, you end up in city B

  ~ RESULT(In(Arad),Go(Zerind)) = In(Zerind))
- **Goal state**
  - Bucharest

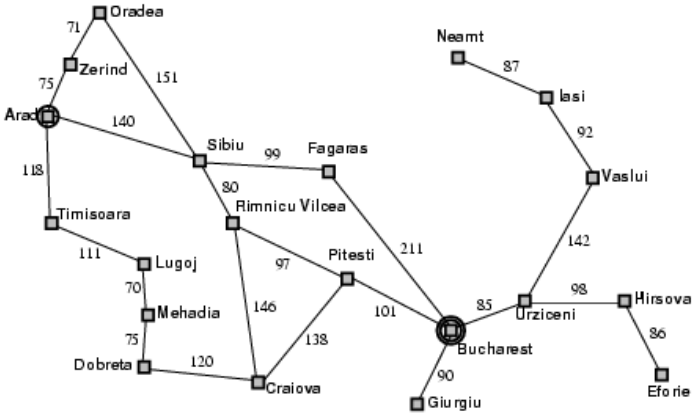
  ~ In(Bucharest)
- **Path cost**
  - Sum of edge costs (total distance traveled)

  ~c(s,a,s')

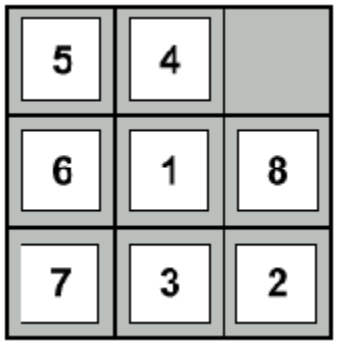


Không gian trạng thái

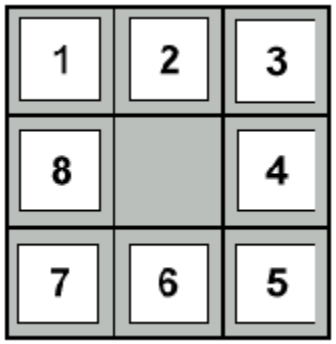
- **The initial state, actions, and transition model define the state space of the problem**
  - The set of all states reachable from initial state by any sequence of actions
  - Can be represented as a **directed graph** where the nodes are states and links between nodes are actions
- **What is the state space for the Romania problem?**



Ví dụ: 8-puzzle



start state

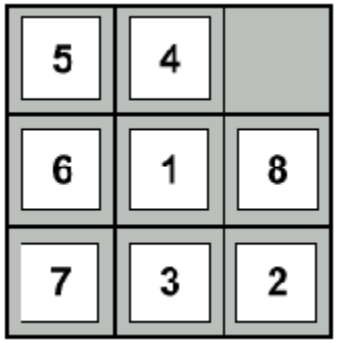


goal state

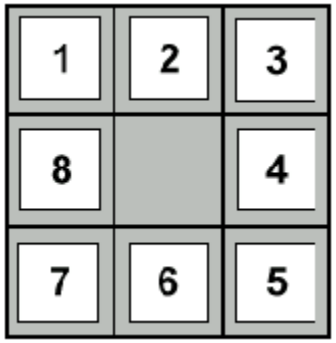
- State :
- Actions :
- Transition model:

- Goal test:
- Path cost:

Ví dụ: 8-puzzle



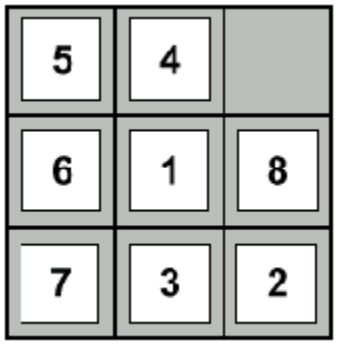
start state



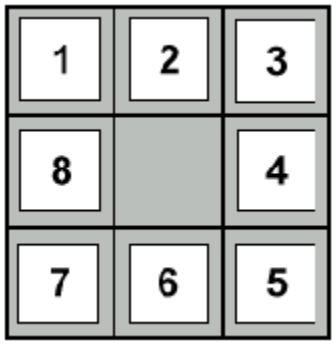
goal state

- State: location of eight tiles and the blank
- Actions: moving the blank space Left, Right, Up, Down
- Transition model: Given a state and action, this returns the resulting state
- Goal test: does state match goal state?
- Path cost: 1 per move

Ví dụ: 8-puzzle



start state



goal state

Why search algorithms?

- 8-puzzle has 9!/2=181,440 states
- 15-puzzle (4x4 board) has 1.3 trillion states states
- 24-puzzle (5x5 board) has about 10^25 states

So, we need a principled way to look for a solution in these huge search spaces…

Tìm kiếm

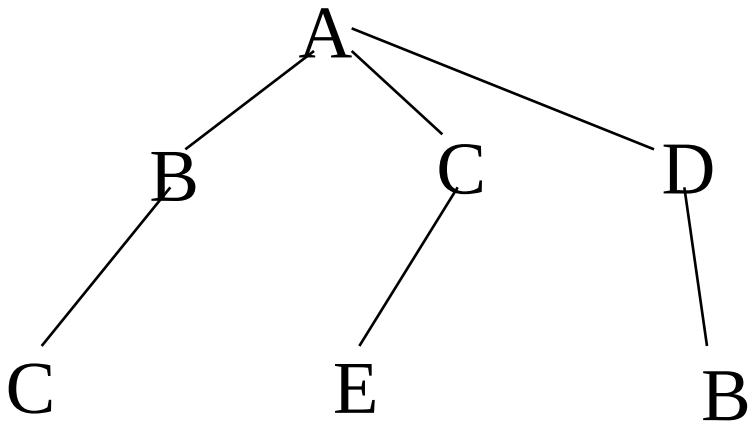
- **Given:**
  - **Initial state**
  - **Actions**
  - **Transition model**
  - **Goal state**
  - **Path cost**
- **How do we find the optimal solution?**
  - How about building the state space and then using Dijkstra's shortest path algorithm?
    - Complexity of Dijkstra's is $O(E + V \log V)$, where V is the size of the state space
    - The state space may be huge!

Ý tưởng

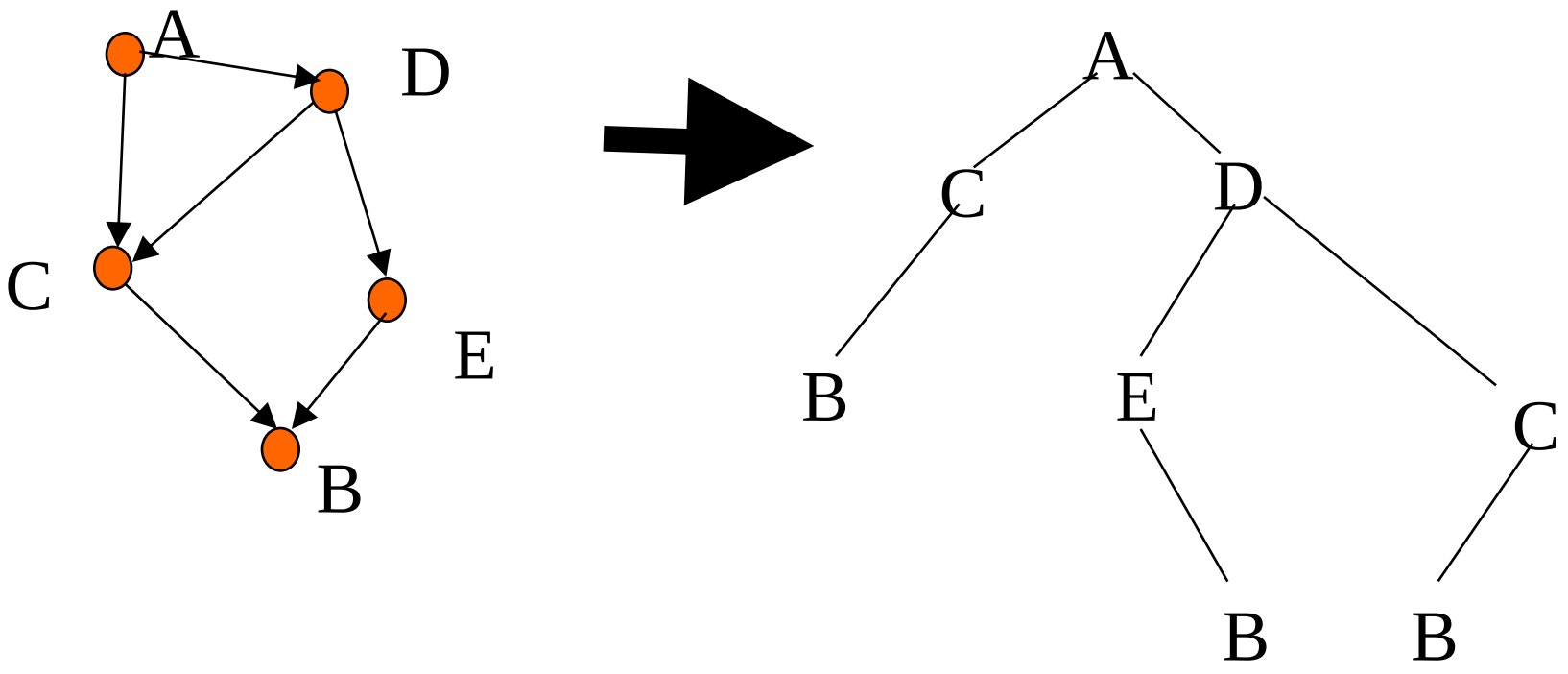
- Let's begin at the start state and **expand** it by making a list of all possible successor states
- Maintain a **frontier** or a list of unexpanded states
- At each step, pick a state from the frontier to expand
- Keep going until you reach a goal state
- Try to expand as few states as possible

Cây tìm kiếm (Search tree)

- Ta có thể thể hiện quá trình tìm kiếm thông qua cây tìm kiếm

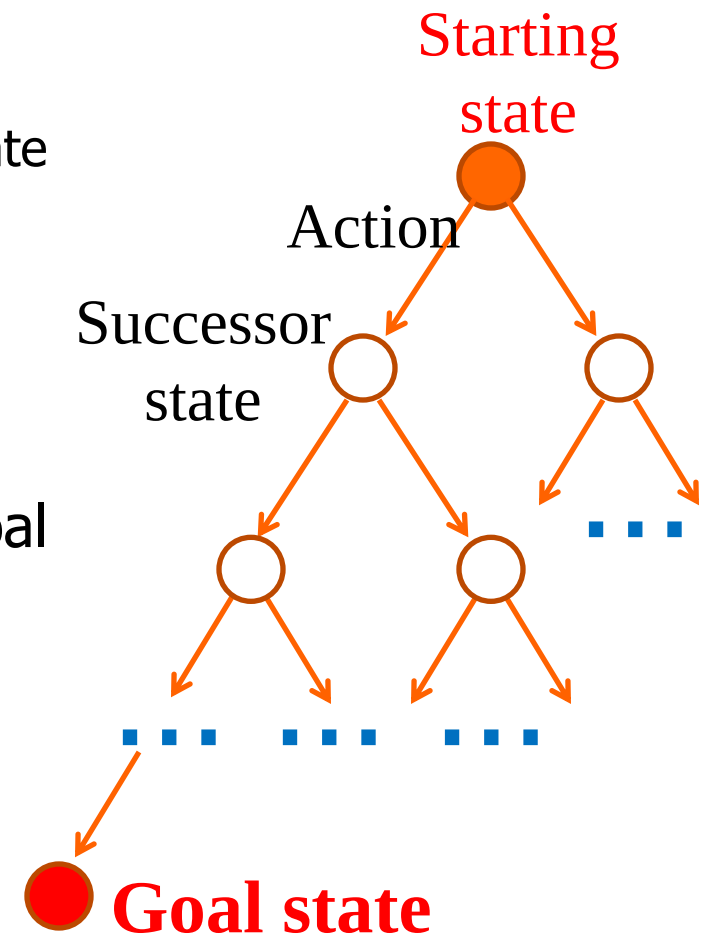


Cây tìm kiếm



Cây tìm kiếm

- "What if" tree of sequences of actions and outcomes
- The root node corresponds to the starting state
- The children of a node correspond to the **successor states** of that node's state
- A path through the tree corresponds to a sequence of actions
  - A solution is a path ending in the goal state
- **Nodes vs. states**
  - A state is a representation of the world, while a node is a data structure that is part of the search tree
    - Node has to keep pointer to parent, path cost, possibly other info



Mô hình thuật toán tìm kiếm theo cây

Function General-Search(*problem*, *strategy*) returns a *solution*, or failure
initialize the search tree using the initial state problem
loop do
if there are no candidates for expansion **then return** failure
choose a leaf node for expansion according to **strategy search**
if the node contains a goal state **then**
return the corresponding solution
else expand the node and add resulting nodes to the search tree
end

Giải quyết vấn đề

- **Problem solving:**
  - Goal formulation
  - Problem formulation (states, operators)
  - Search for solution

- **Problem formulation:**
  - Initial state
  - Actions
  - Goal test
  - Path cost

Tìm kiếm giải pháp

Solution: is ???

Function General-Search(*problem*, *strategy*) returns a *solution*, or failure
    initialize the search tree using the initial state problem
    loop do
        if there are no candidates for expansion **then return** failure
        choose a leaf node for expansion according to strategy
        if the node contains a goal state **then return** the corresponding solution
        else expand the node and add resulting nodes to the search tree
    end

Tìm kiếm giải pháp

Solution: is a sequence of actions that bring you from current state to the goal state.

Function General-Search(*problem*, *strategy*) returns a *solution*, or failure
    initialize the search tree using the initial state problem
    loop do
        if there are no candidates for expansion **then return** failure
        choose a leaf node for expansion according to strategy
        if the node contains a goal state **then return** the corresponding solution
        else expand the node and add resulting nodes to the search tree
    end

Strategy: The search strategy is determined by ???

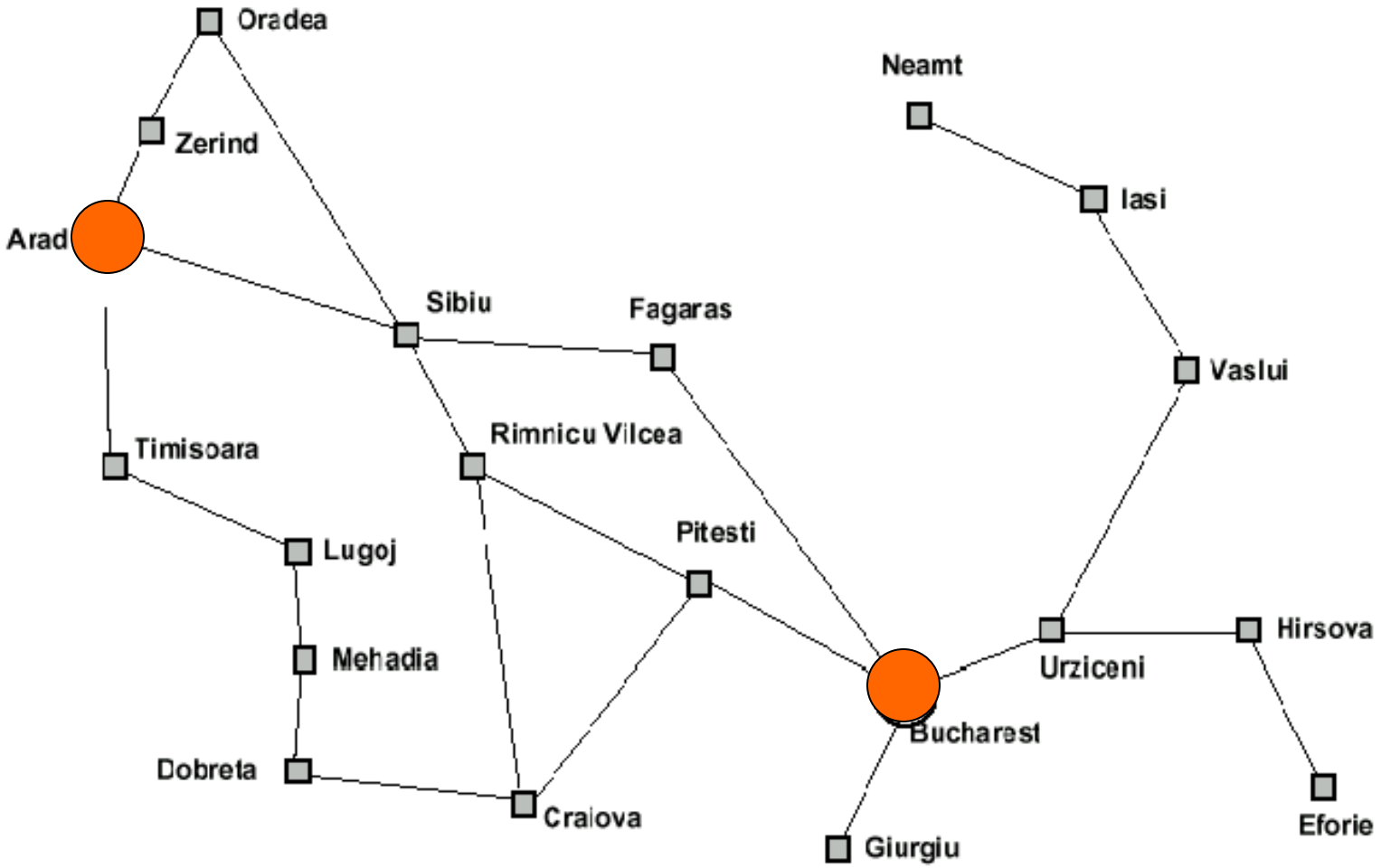
Tìm kiếm giải pháp

Solution: is a sequence of actions that bring you from current state to the goal state

Function General-Search(*problem*, ***strategy***) returns a *solution*, or failure
    initialize the search tree using the initial state problem
    loop do
        if there are no candidates for expansion **then return** failure
        choose a leaf node for expansion according to strategy
        if the node contains a goal state **then return** the corresponding solution
        else expand the node and add resulting nodes to the search tree
    end

Strategy: The search strategy is determined by the **order** in which the nodes are expanded.

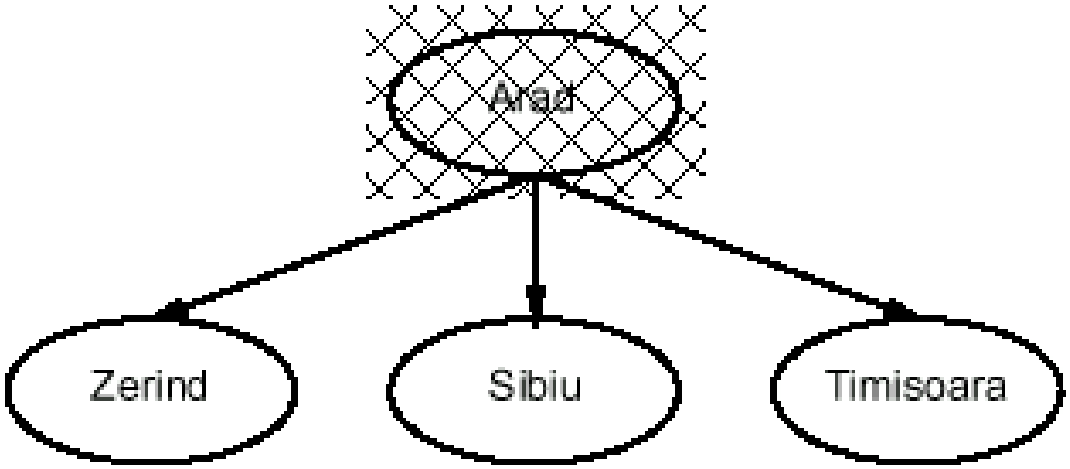
Ví dụ: Đi từ Arad tới Bucharest



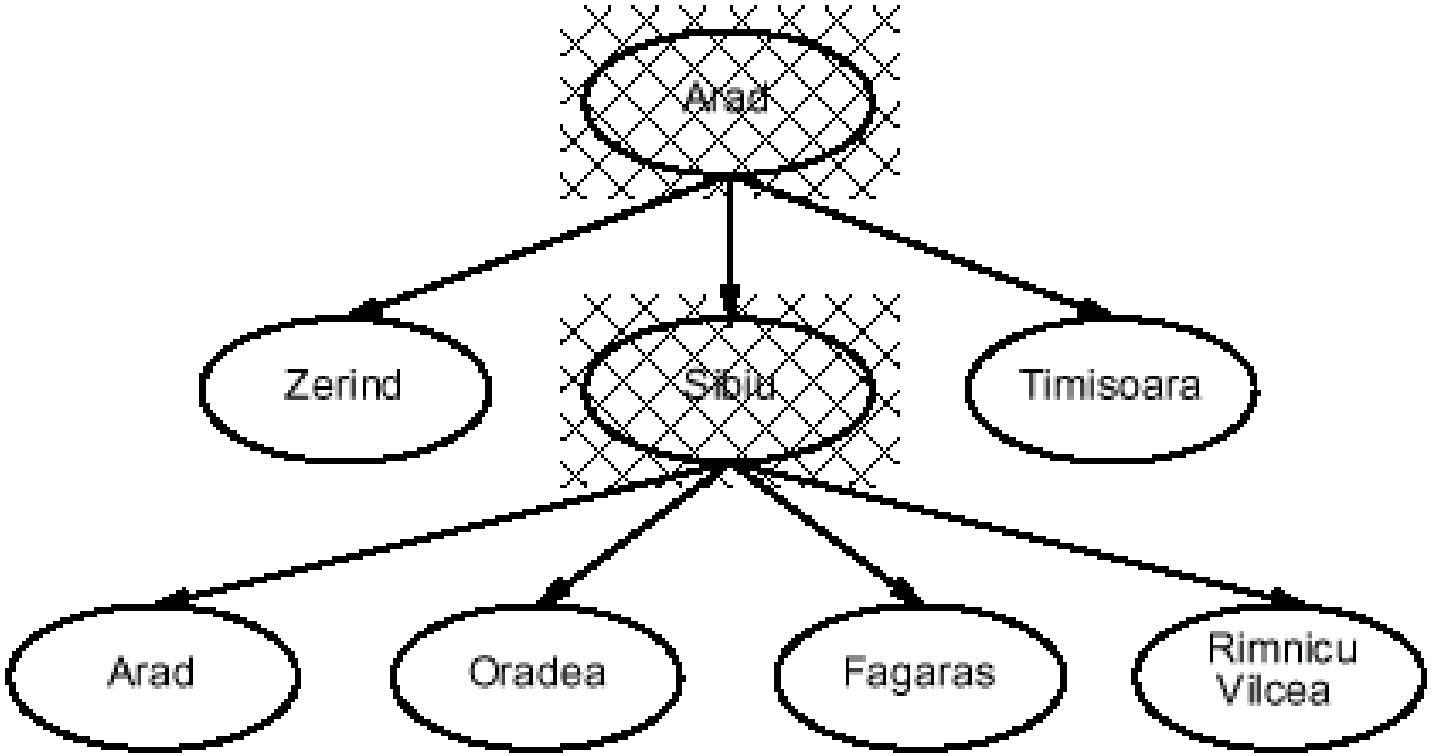
Ví dụ cây tìm kiếm



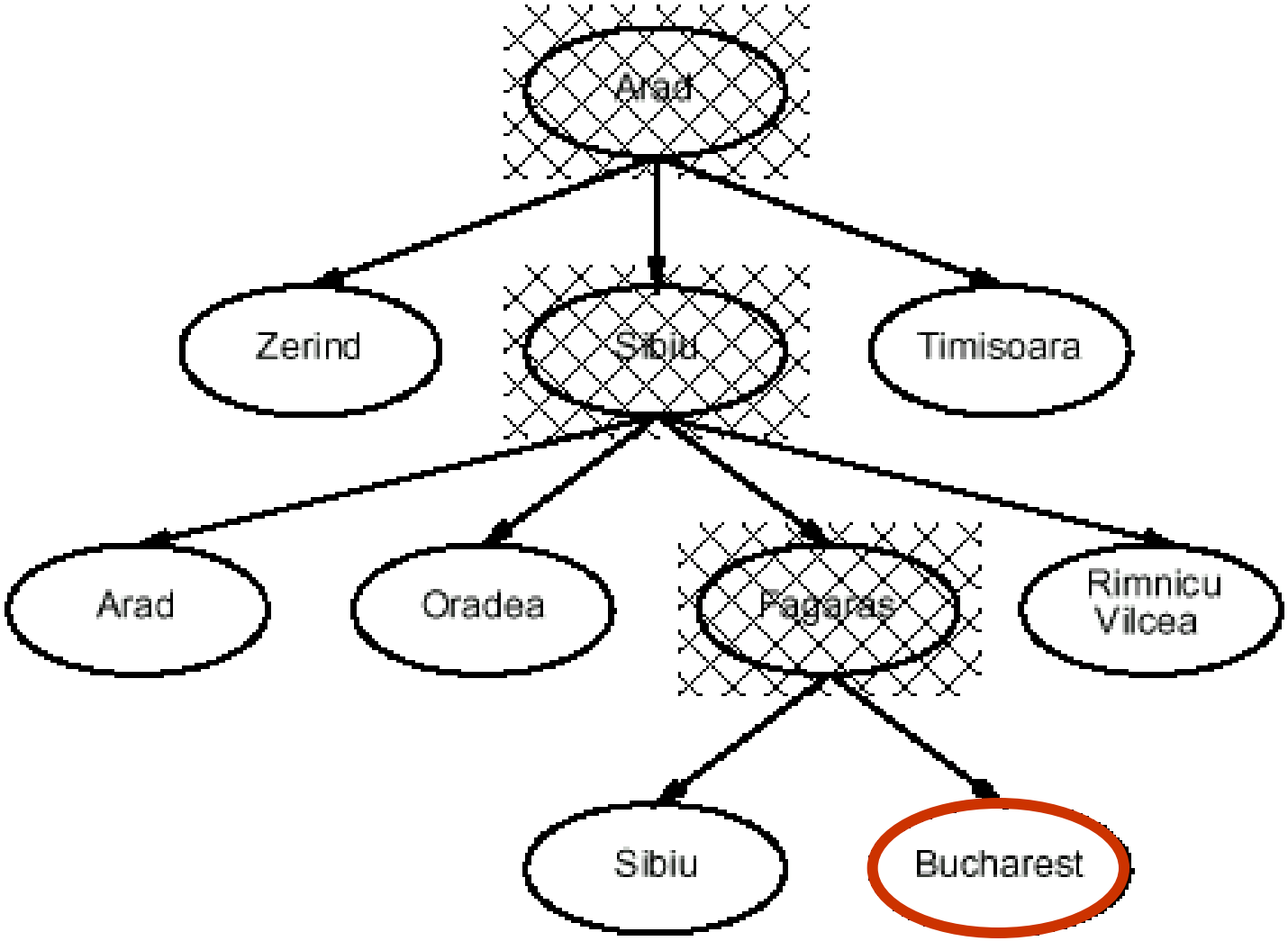
Ví dụ cây tìm kiếm



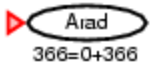
Ví dụ cây tìm kiếm



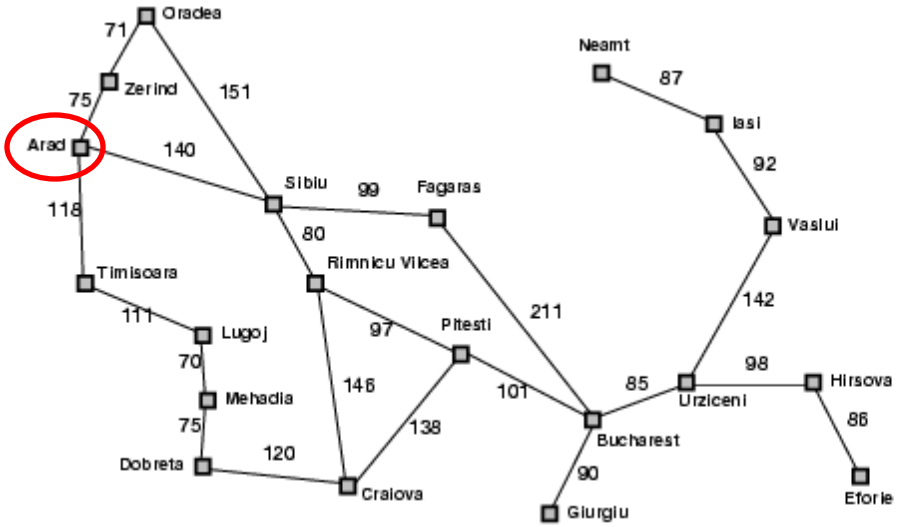
Ví dụ cây tìm kiếm



Ví dụ cây tìm kiếm



Start: Arad
Goal: Bucharest



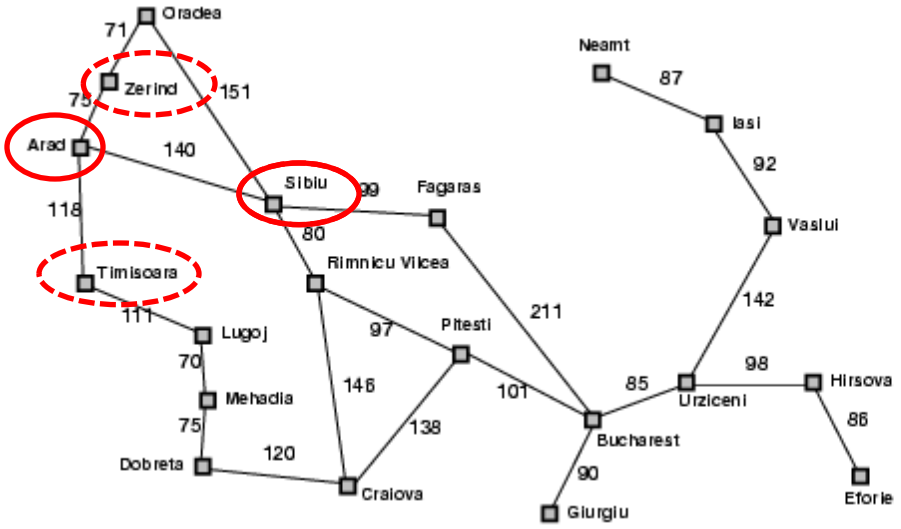
Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Ví dụ cây tìm kiếm



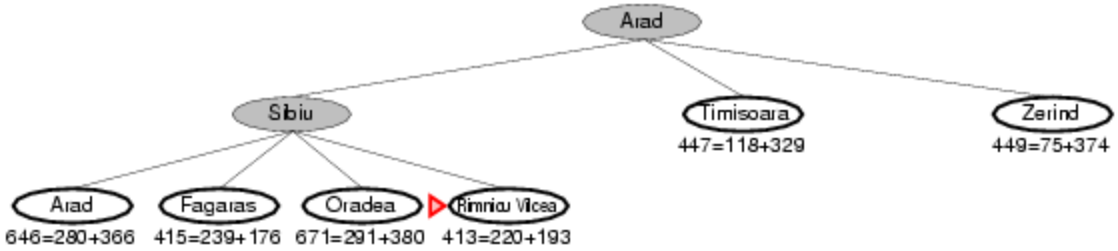
Start: Arad

Goal: Bucharest



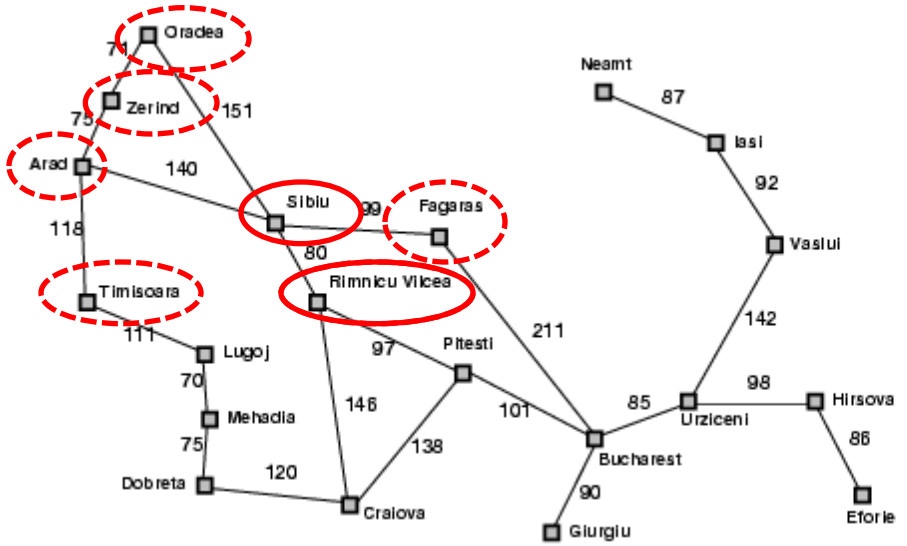
Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Ví dụ cây tìm kiếm



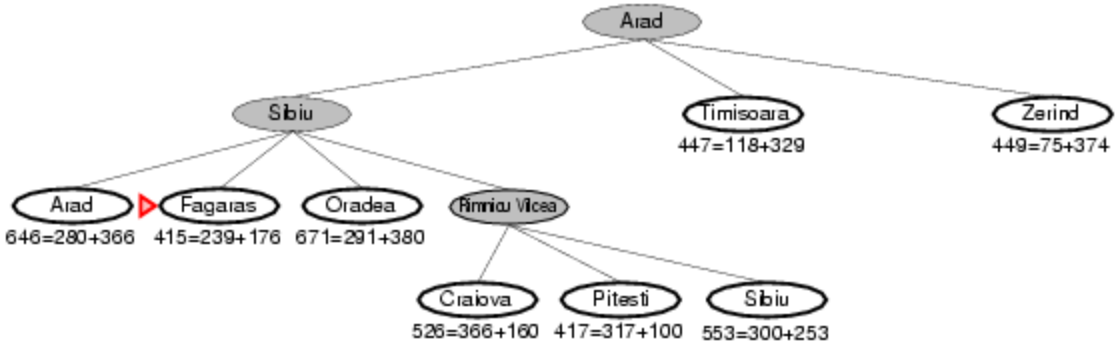
Start: Arad

Goal: Bucharest



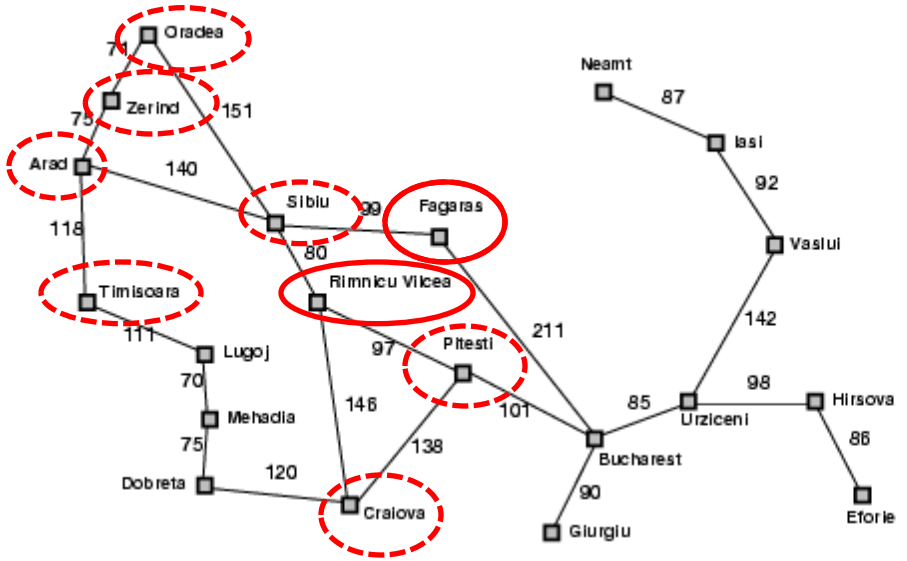
Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Ví dụ cây tìm kiếm



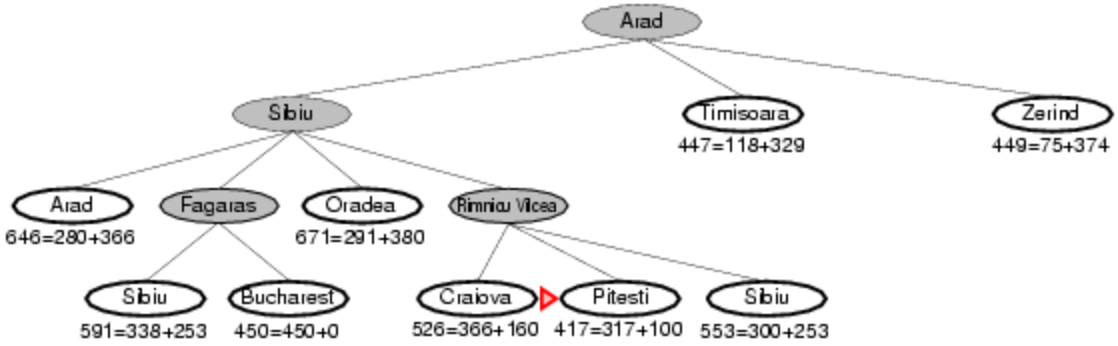
Start: Arad

Goal: Bucharest



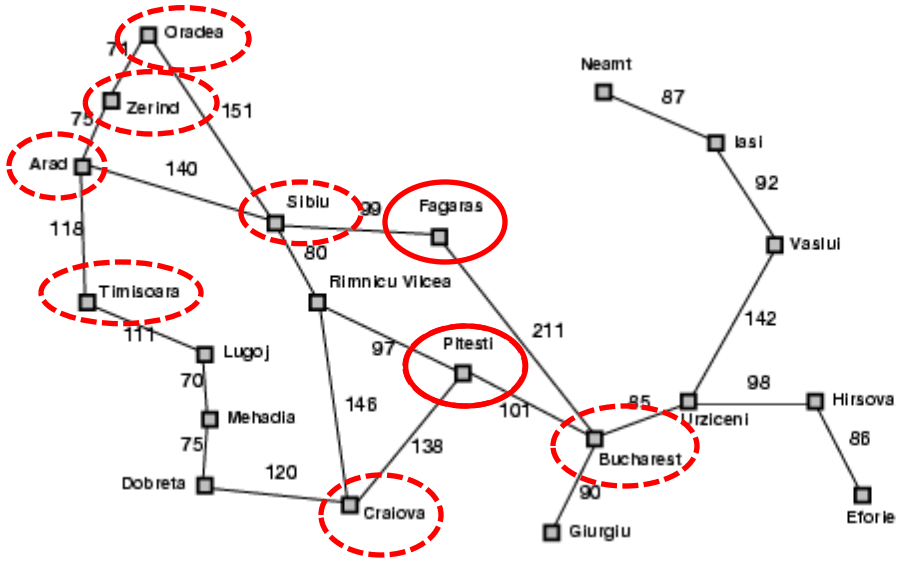
Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Ví dụ cây tìm kiếm



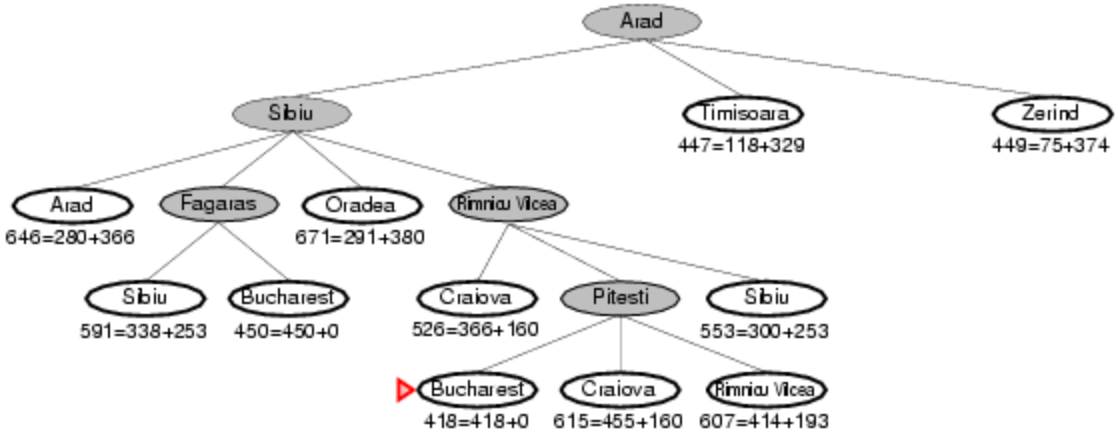
Start: Arad

Goal: Bucharest



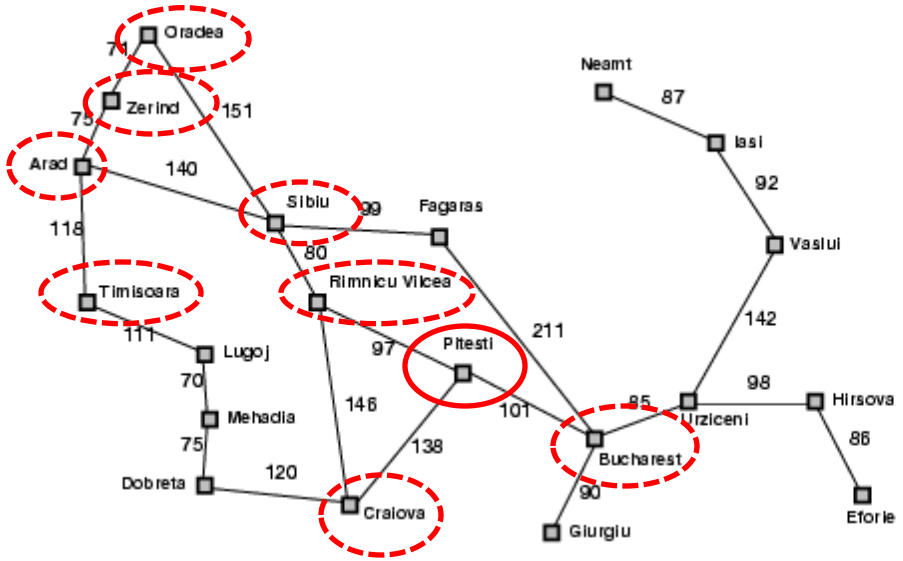
Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Ví dụ cây tìm kiếm



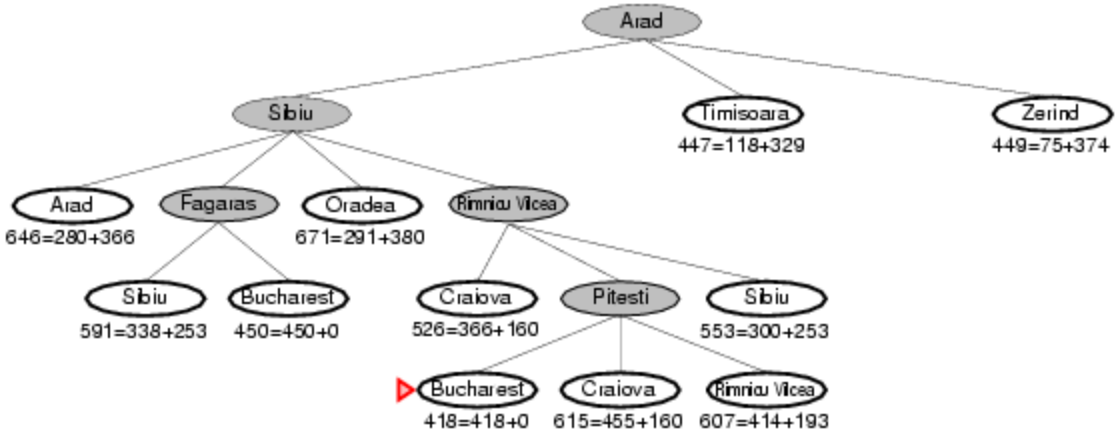
Start: Arad

Goal: Bucharest



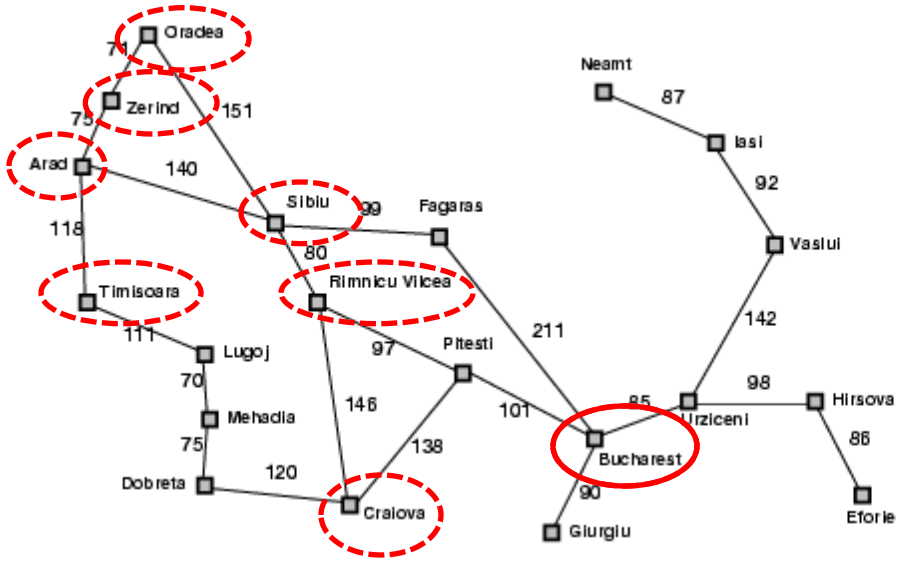
Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Ví dụ cây tìm kiếm



Start: Arad

Goal: Bucharest



Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Quản lý các trạng thái lặp

- **To handle repeated states:**
  - Every time you expand a node, add that state to the **explored set**; do not put explored states on the frontier again
  - Every time you add a node to the frontier, check whether it already exists in the frontier with a higher path cost, and if yes, replace that node with the new one

Thực hiện các thuật toán tìm kiếm

```
Function General-Search(problem, Queuing-Fn) returns a solution, or failure
    nodes ← make-queue(make-node(initial-state[problem]))
    loop do
        if nodes is empty then return failure
        node ← Remove-Front(nodes)
        if Goal-Test[problem] applied to State(node) succeeds then return node
        nodes ← Queuing-Fn(nodes, Expand(node, Operators[problem]))
    end
```

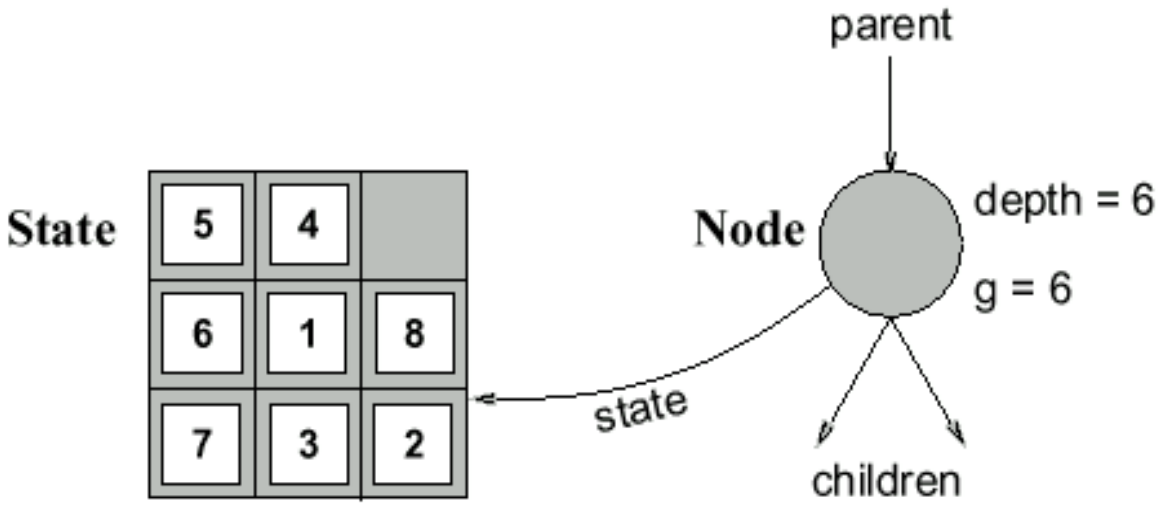
Queuing-Fn(*queue, elements*) is a queuing function that inserts a set of elements into the queue and determines the order of node expansion. Varieties of the queuing function produce varieties of the search algorithm.

Encapsulating *state* information in *nodes*

A *state* is a (representation of) a physical configuration

A *node* is a data structure constituting part of a search tree
includes *parent*, *children*, *depth*, *path cost* $g(x)$

States do not have parents, children, depth, or path cost!



The EXPAND function creates new nodes, filling in the various fields and using the OPERATORS (or SUCCESSORFN) of the problem to create the corresponding states.

Các tiêu chí đánh giá các chiến lược tìm kiếm

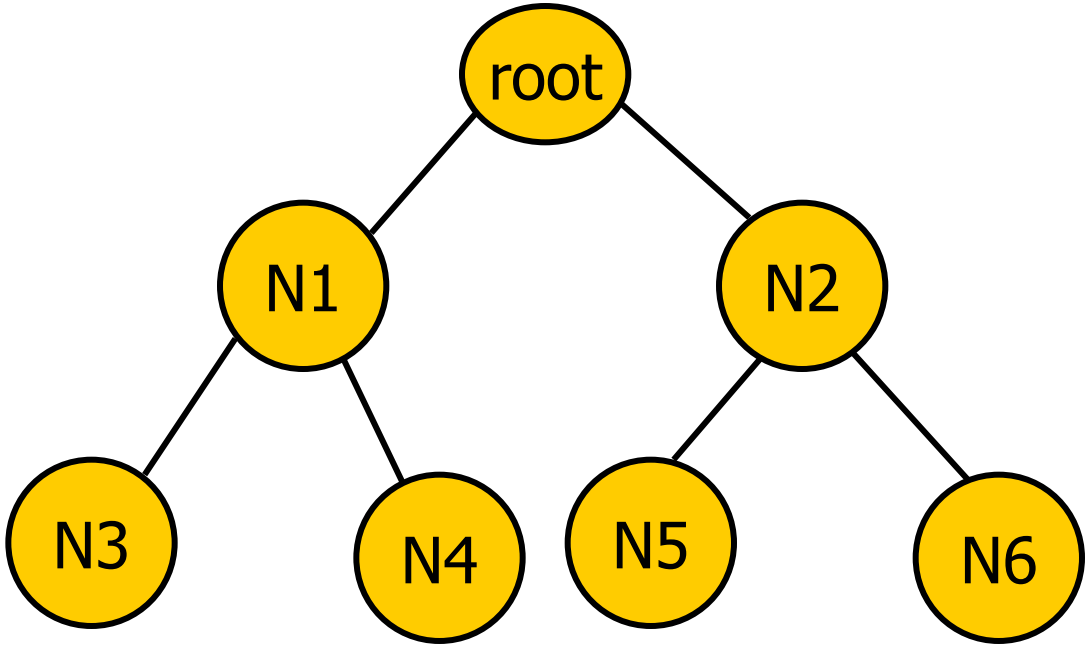
- A search strategy is defined by picking the order of node expansion (Một chiến lược tìm kiếm được định nghĩa bằng chọn thứ tự của việc mở rộng nút)
- Completeness (độ hoàn chỉnh): Có phải lúc nào cũng tìm ra nghiệm nếu có không?
- Time complexity (độ phức tạp về thời gian): số lượng các trạng thái đã đến
- Space complexity (độ phức tạp về không gian): số lượng các trạng thái tối đa phải lưu trong bộ nhớ
- Optimality (độ tối ưu): có phải lúc nào cũng tìm ra nghiệm có chi phí nhỏ nhất không?
- Các tham số để tính độ phức tạp về không gian và thời gian:
  - *b:* maximum branching factor of the search tree (số trạng thái kề)
  - *d:* depth of the least-cost solution (độ sâu của trạng thái kết thúc)
  - *m*: maximum depth of the state space (may be ∞)(độ sâu lớn nhất)

Ví dụ cây tìm kiếm nhị phân

Depth = 0

Depth = 1

Depth = 2



Number of nodes: n = $2^{\text{max depth}}$

Number of levels (max depth) = log(n) (could be n)

Độ phức tạp

- Why worry about complexity of algorithms?

➢ because a problem may be solvable in principle but may take too long to solve in practice

Độ phức tạp: Tháp Hà Nội

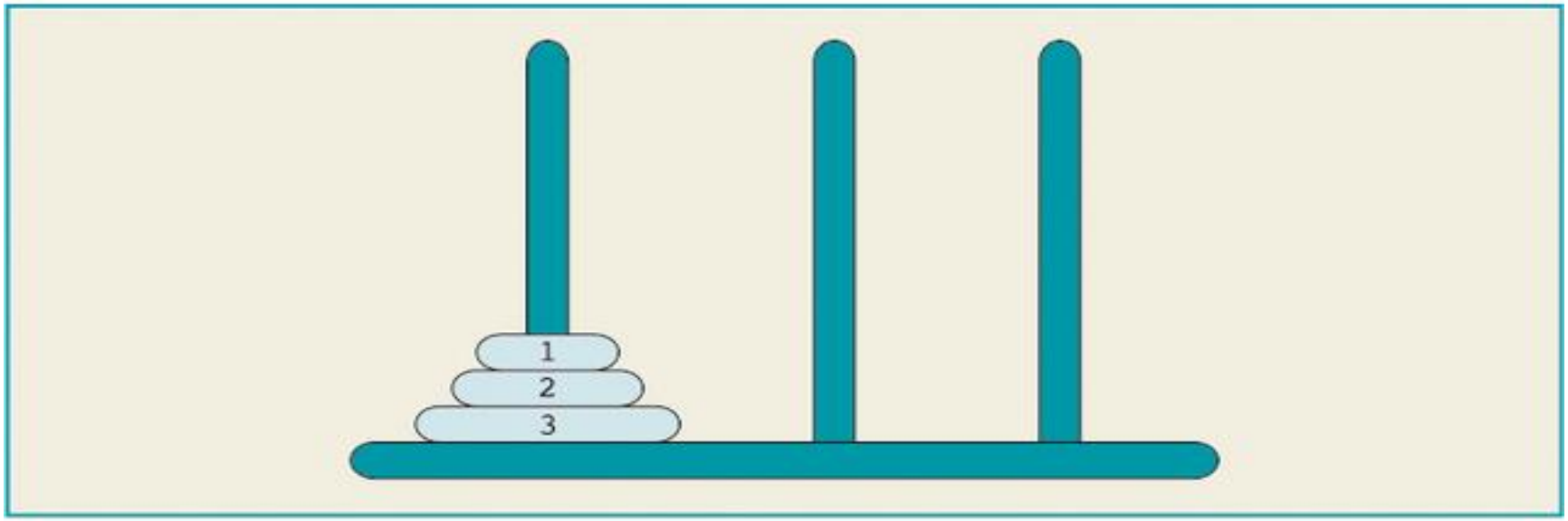


Figure 11-6 Tower of Hanoi problem with three disks

Độ phức tạp: Tháp Hà Nội

"THÁP HÀ NỘI
Trò chơi trí tuệ của An nam
Trò chơi được đem về từ Bắc Kì
Bởi giáo sư N. CLAUS (của SIAM)
Trường Cao đẳng Quan Li-Sou-Stian!

....

LA TOUR D'HANOÏ

VERITABLE CASSE-TÊTE ANNAMITE

JEU RAPPORTÉ DU TONKIN

PAR LE PROFESSEUR N. CLAUS (DE SIAM)

Mandarin du Collège Li-Sou-Stian!

Ce jeu inédit a été trouvé, pour la première fois, dans les écrits de l'illustre Mandarin FER-FER-TAM-TAM, qui seront publiés, plus ou moins prochainement, suivant les ordres du Gouvernement chinois.

La TOUR D'HANOÏ se compose d'étages superposés et décroissants, en nombre variable, que nous avons représentés par huit pions en bois, percés à leur centre. Au Japon, en Chine, au Tonkin, on les fait en porcelaine.

Le jeu consiste à démolir la tour, étage par étage, et à la reconstruire dans un lieu voisin conformément aux règles indiquées.

Amusant et instructif, facile à apprendre et à jouer, à la ville, à la campagne, en voyage, il a pour but la vulgarisation des sciences, comme tous les autres jeux curieux et inédits du professeur N. CLAUS (DE SIAM).

Nous pourrions offrir une *prime de dix mille francs*, de *cent mille francs*, d'*un million de francs*, et plus encore, à celui qui réaliserait, à la main, le transport de la Tour d'Hanoï, à soixante-quatre étages, conformément aux règles du jeu. Nous dirons, tout de suite, qu'il faudrait exécuter successivement le nombre de déplacements

18 446 744 073 709 551 615,

ce qui exigerait plus de *cinq milliards de siècles!*

D'après une vieille légende indienne, les brahmes se succèdent depuis bien longtemps, sur les marches de l'autel, dans le Temple de Bénarès, pour exécuter le déplacement de la *Tour Sacrée*, de Brahma, aux soixante-quatre étages en or fin, garnis de diamants de Golconde. Quand tout sera fini, la Tour et les brahmes tomberont, et ce sera la fin du monde!

PARIS, PÉKIN, YÉDO et SAÏGON

Chez les libraires et marchands de nouveautés.

1883

Tous droits réservés.

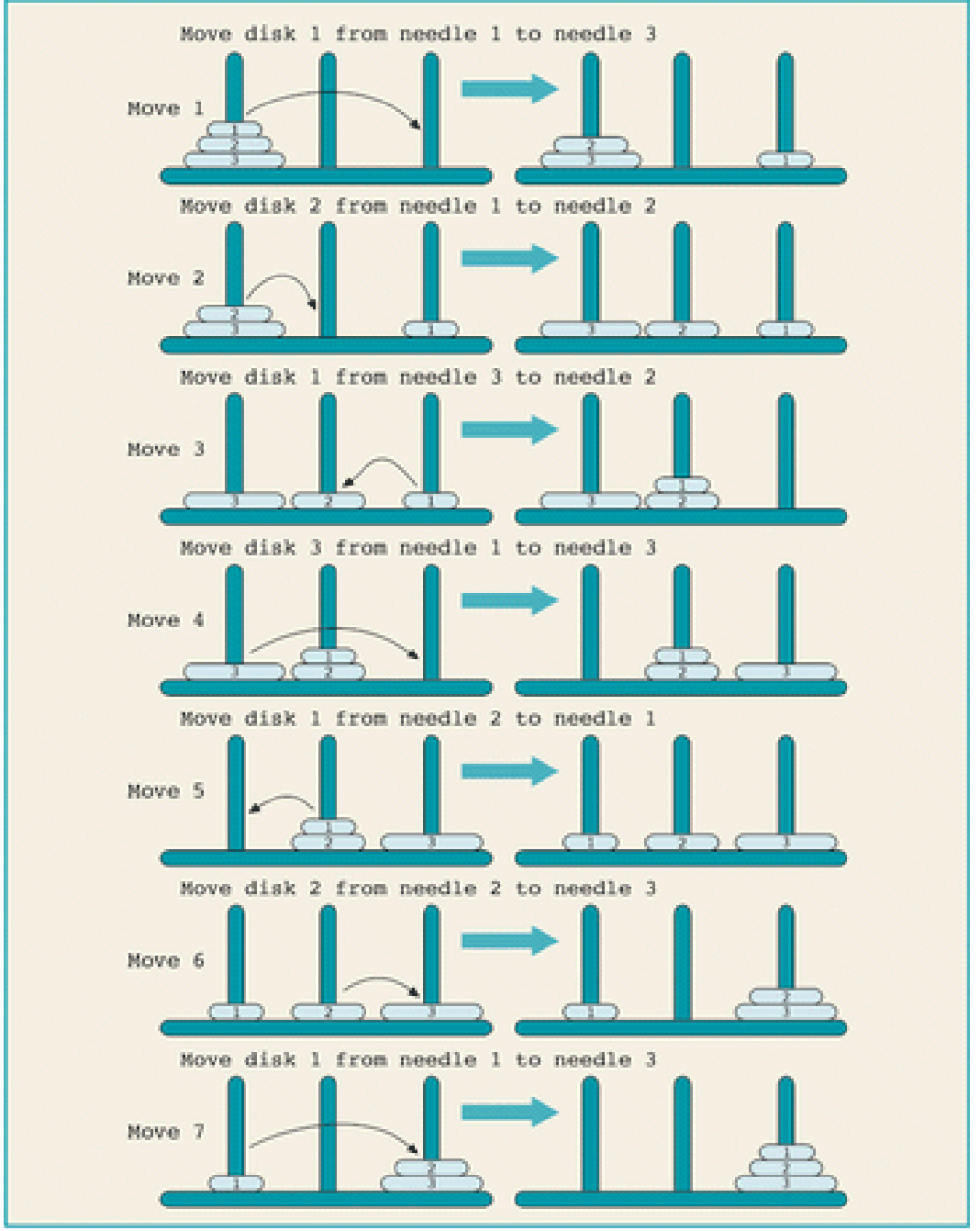


Figure 11-7 Solution of Tower of Hanoi problem with three disks

Độ phức tạp: Tháp Hà Nội

- 3-disk problem: $2^3 - 1 = 7$ moves
- 64-disk problem: $2^{64} - 1$.
  - $2^{10} = 1024 \approx 1000 = 10^3$,
  - $2^{64} = 2^4 * 2^{60} \approx 2^4 * 10^{18} = 1.6 * 10^{19}$
- One year $\approx 3.2 * 10^7$ seconds

Độ phức tạp: Tháp Hà Nội

- The wizard's speed = one disk / second

$$1.6 * 10^{19} = 5 * 3.2 * 10^{18} =$$
$$5 * (3.2 * 10^{7}) * 10^{11} =$$
$$(3.2 * 10^{7}) * (5 * 10^{11})$$

Độ phức tạp: Tháp Hà Nội

- The time required to move all 64 disks from needle 1 to needle 3 is roughly $5 * 10^{11}$ years.

- It is estimated that our universe is about 15 billion $= 1.5 * 10^{10}$ years old.

$$5 * 10^{11} = 50 * 10^{10} \approx 33 * (1.5 * 10^{10}).$$

Độ phức tạp: Tháp Hà Nội

- Assume: a computer* with 1 billion = 10^9 moves/second.
  - **Moves/year=(3.2 *10^7) * 10^9 = 3.2 * 10^{16}**

- To solve the problem for 64 disks:
  - $2^{64} \approx 1.6 * 10^{19} = 1.6 * 10^{16} * 10^3 =$

    $(3.2 * 10^{16}) * 500$

  - 500 years for the computer to generate 2^{64} moves at the rate of 1 billion moves per second.

The world's fastest computer (getting close to petaflopspeed) is IBM's Blue Gene with a top speed of 360 trillion operations a second.

Độ phức tạp

- Why worry about complexity of algorithms?

➢ because a problem may be solvable in principle but may take too long to solve in practice

- How can we evaluate the complexity of algorithms?

➢ through asymptotic (tiệm cận) analysis, i.e., estimate time (or number of operations) necessary to solve an instance of size *n* of a problem when *n* tends towards infinity

➢ See AIMA, Appendix A.



SEE YOU IN THE NEXT WEEK !

Nhớ lại: Thực hiện các thuật toán tìm kiếm

```
Function General-Search(problem, Queuing-Fn) returns a solution, or failure
    nodes ← make-queue(make-node(initial-state[problem]))
    loop do
        if nodes is empty then return failure
        node ← Remove-Front(nodes)
        if Goal-Test[problem] applied to State(node) succeeds then return node
        nodes ← Queuing-Fn(nodes, Expand(node, Operators[problem]))
    end
```

Queuing-Fn(*queue, elements*) is a queuing function that inserts a set of elements into the queue and determines the order of node expansion. Varieties of the queuing function produce varieties of the search algorithm.

Các loại chiến lược tìm kiếm

- Các chiến lược tìm kiếm mù (uninformed/blind search):
  - Không hiểu biết gì về đối tượng
  - Duyệt qua để tìm được đối tượng cần tìm
- Các chiến lược tìm kiếm kinh nghiệm (informed/heuristic search):
  - Dựa vào kinh nghiệm và sự hiểu biết của chúng ta để xây dựng hàm đánh giá hướng dẫn tìm kiếm
- Các chiến lược tìm kiếm tối ưu
- Các phương pháp tìm kiếm có đối thủ

Uninformed/Blind search strategies (Các chiến lược tìm kiếm mù)

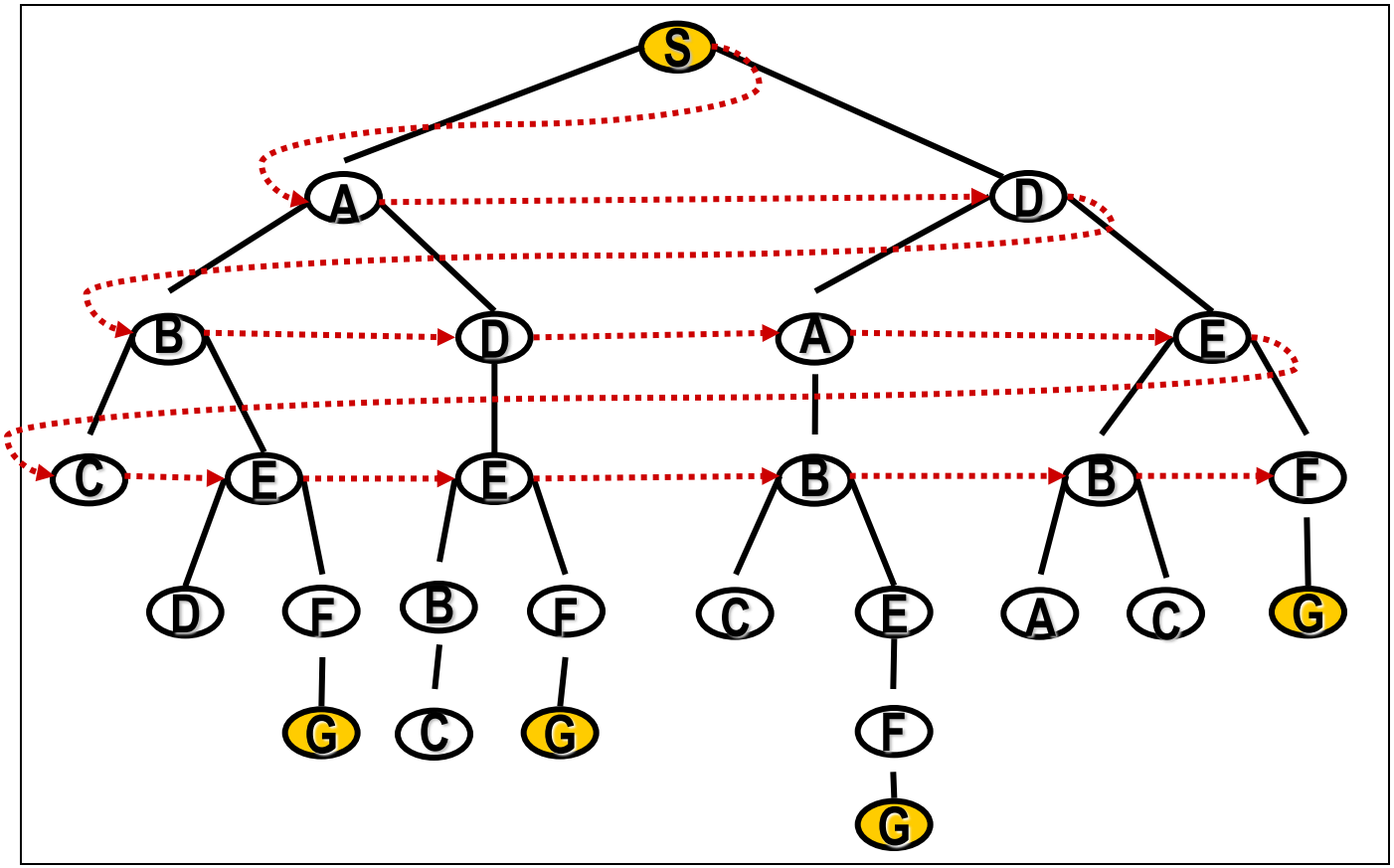
Các chiến lược tìm kiếm mù

- Tìm kiếm theo bề rộng (breadth-first search)
- Tìm kiếm theo chiều sâu (depth-first search)
- Tìm kiếm theo chiều sâu có giới hạn (depth-limited search)
- Tìm kiếm theo chiều sâu lặp (iterative deepening depth-first search)

Tìm kiếm theo chiều rộng (BFS) - Ý tưởng

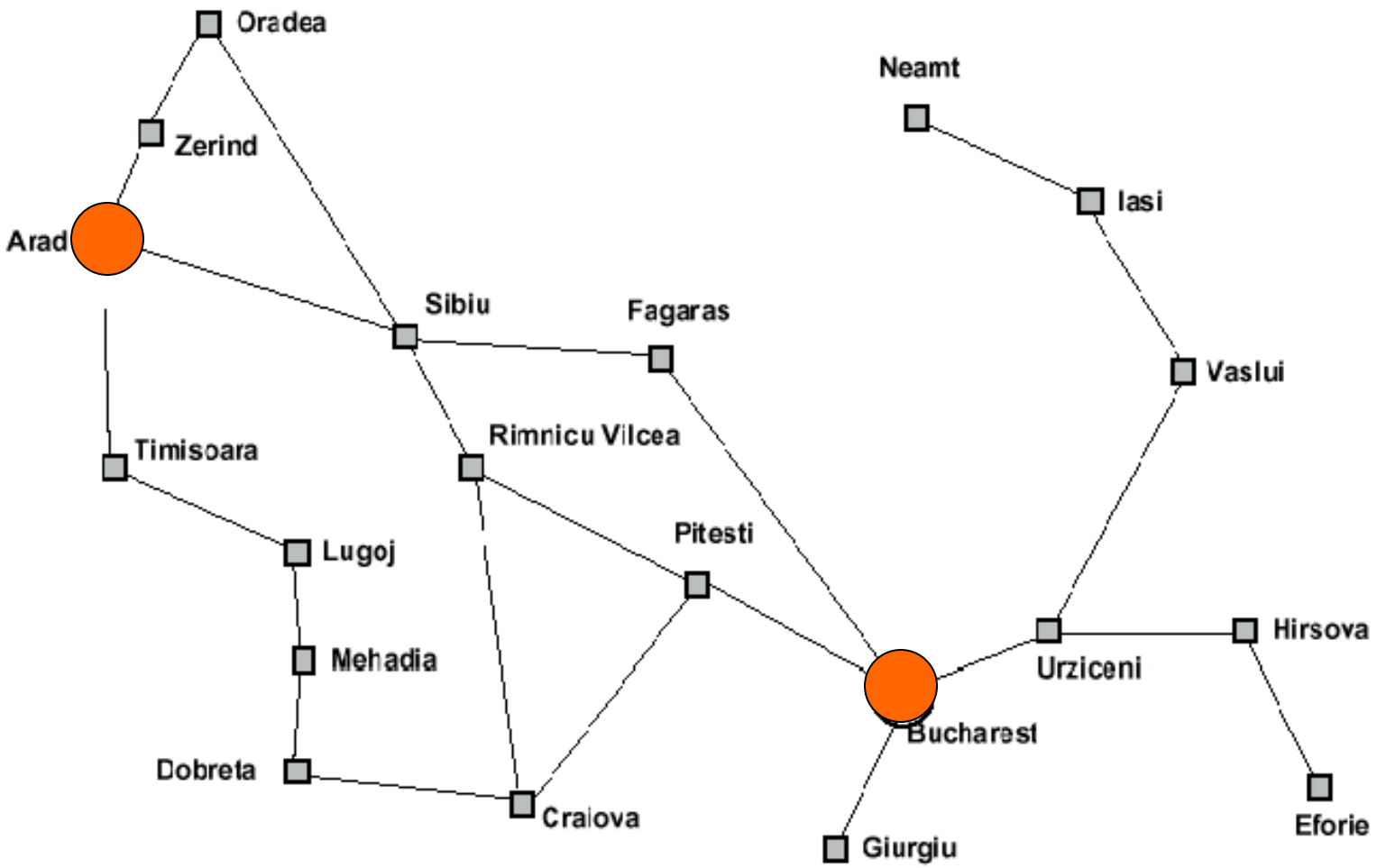
- Tại mỗi bước, chọn trạng thái để phát triển là trạng thái được sinh ra trước các trạng thái chờ phát triển khác
- Nói một cách khác, chúng ta sẽ quét cây tìm kiếm theo từng lớp tính từ gốc trở đi
- Đỉnh mới được sinh ra sẽ được lưu vào một danh sách tổ chức theo kiểu "hàng đợi" ***(queue FIFO)***

BFS – Ý tưởng

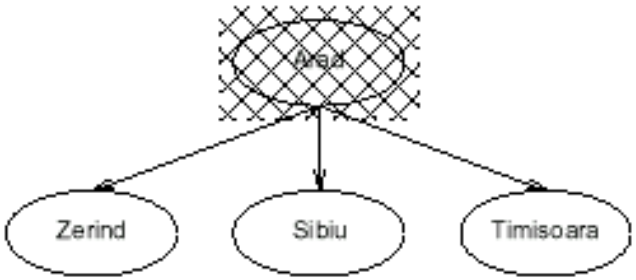


Move downwards, level by level, until goal is reached.

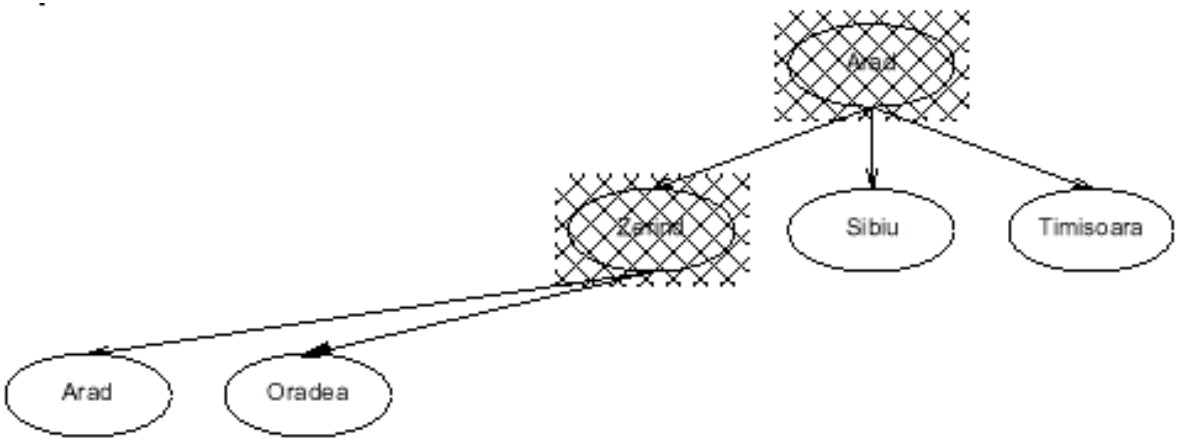
Ví dụ: Đi từ Arad tới Bucharest



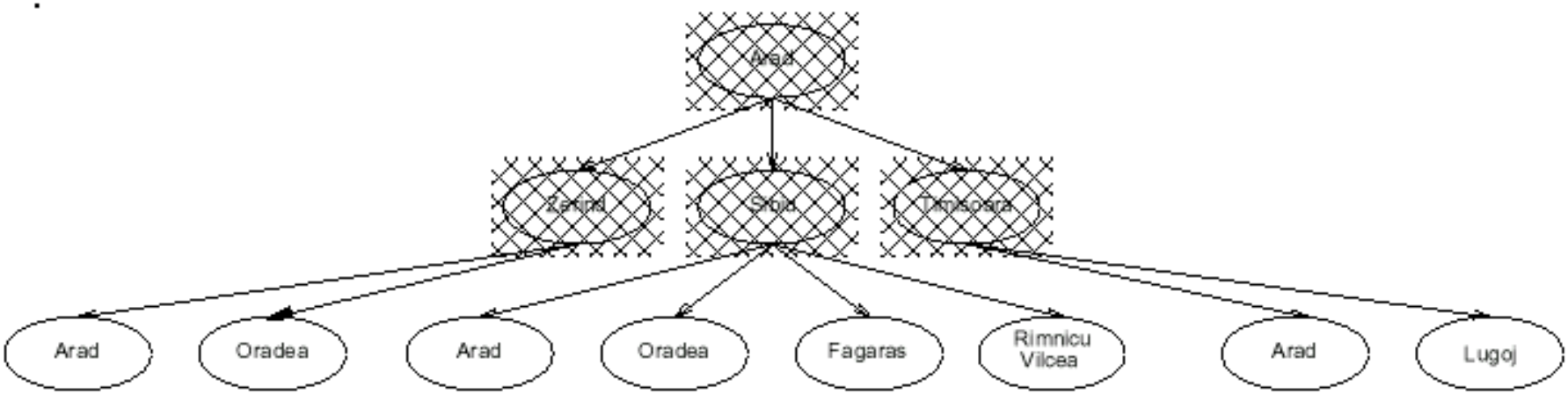
Ví dụ



Ví dụ



Ví dụ


Arad
Zerind
Sibiu
Timisoara
Arad
Oradea
Arad
Oradea
Fagaras
Rimnicu Vilcea
Arad
Lugoj

Mô tả thuật toán BFS

Từ thuật toán tìm kiếm tổng quát ta có:

1. Khởi tạo hàng đợi L chỉ chứa trạng thái ban đầu
2. **Loop do**
   - 2.1 **If** (L rỗng) **then**
     {thông báo tìm kiếm thất bại; kết thúc}
   - 2.2 Loại trạng thái u ở đầu hàng đợi L;
   - 2.3 **If** (u là trạng thái kết thúc) **then**
     {thông báo tìm kiếm thành công; kết thúc}
   - 2.4 **For** (mỗi trạng thái v kề u) **do**
     {Đặt v vào cuối hàng đợi L;}

Tính chất của thuật toán BFS

- Completeness:
- Time complexity:
- Space complexity:
- Optimality:

- Search algorithms are commonly evaluated according to the following four criteria:
  - **Completeness:** does it always find a solution if one exists?
  - **Time complexity:** how long does it take as function of num. of nodes?
  - **Space complexity:** how much memory does it require?
  - **Optimality:** does it guarantee the least-cost solution?

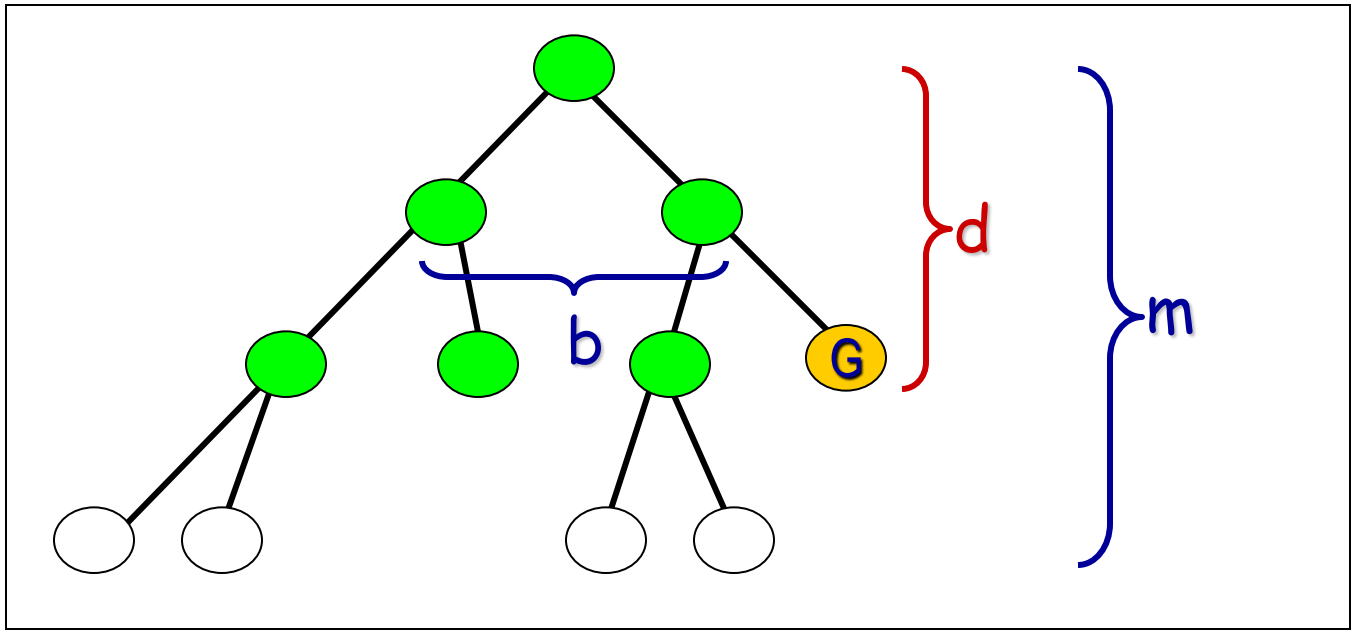
- Time and space complexity are measured in terms of:
  - *b* – max branching factor of the search tree
  - *d* – depth of the least-cost solution
  - *m* – max depth of the search tree (may be infinity)

Tính chất của thuật toán BFS

- Completeness: Yes, if *b* is finite
- Time complexity: $1+b+b^2+\ldots+b^d = O(b^d)$, i.e., exponential in *d*
- Space complexity: $O(b^d)$ (see following slides)
- Optimality: Yes (assuming cost = 1 per step)

Độ phức tạp thời gian của BFS

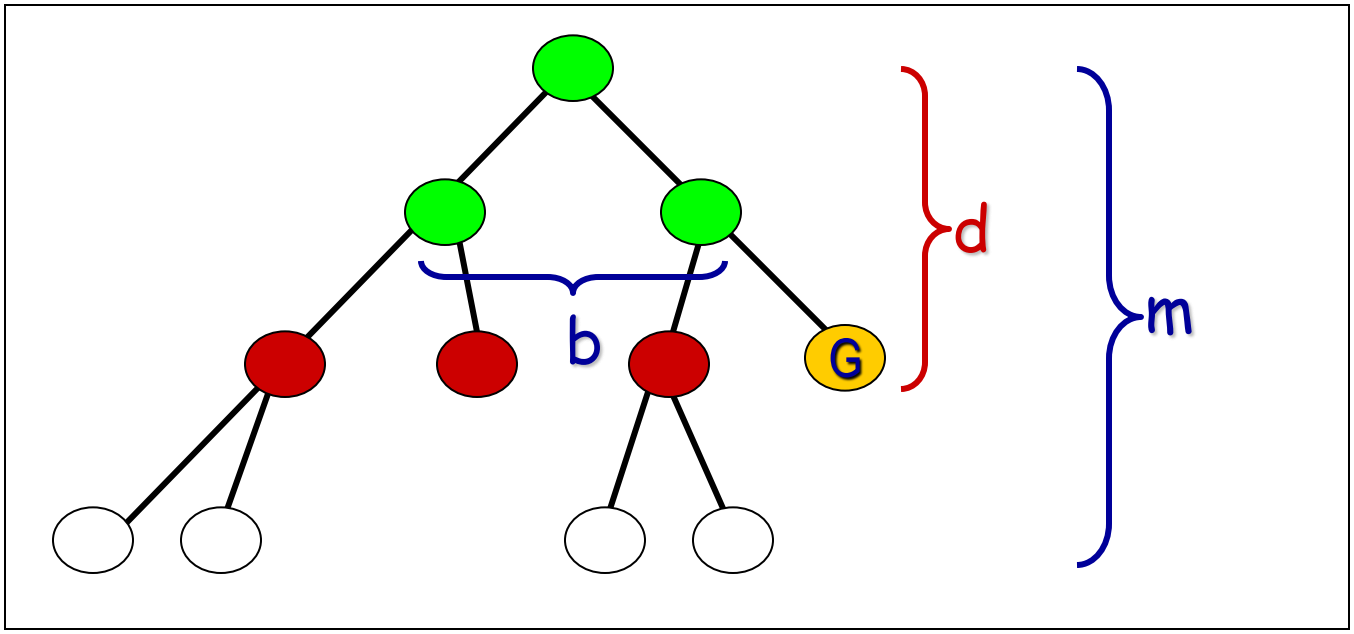
- If a goal node is found on depth d of the tree, all nodes up till that depth are created.



- Thus: $O(b^d)$

Độ phức tạp không gian của BFS

- Largest number of nodes in QUEUE is reached on the level d of the goal node.



- QUEUE contains all  and  nodes. (Thus: 4) .
- In General: b^d

Ví dụ về độ phức tạp thời gian và không gian của BFS

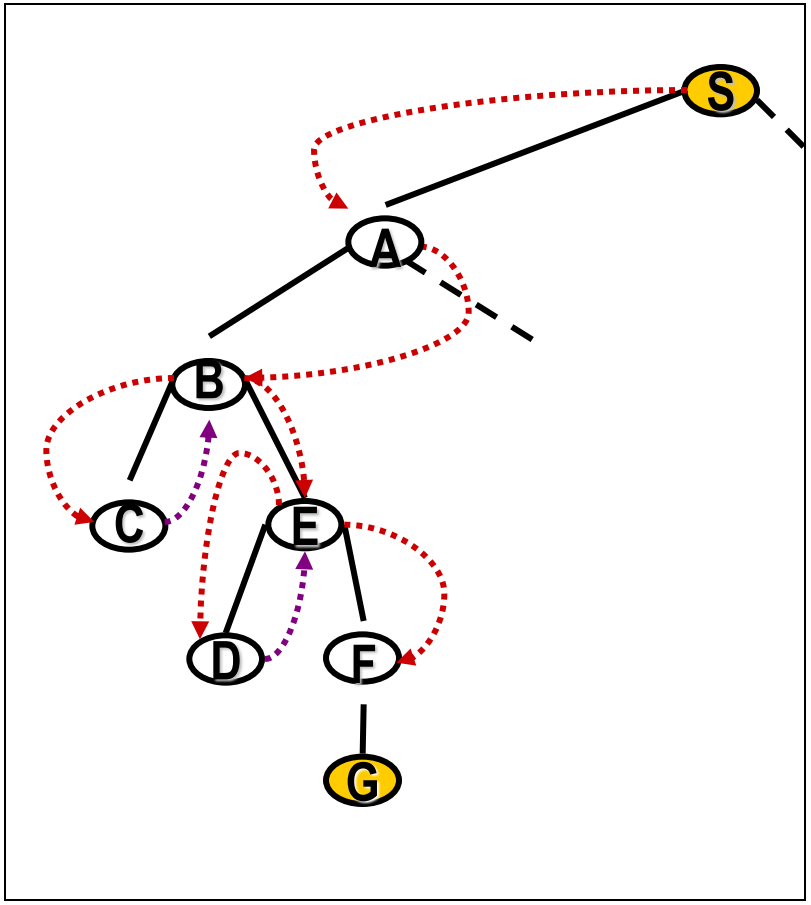
- Giả sử b=10, kiểm tra 1000 trạng thái cần 1 giây, lưu một trạng thái cần 100 byte

Độ sâu d	Thời gian	Không gian
4	11 giây	1 megabyte
6	18 phút	111 megabyte
8	31 giờ	11 gigabyte
10	128 ngày	1 terabyte
12	35 năm	111 terabyte
14	3500 năm	11 111 terabyte

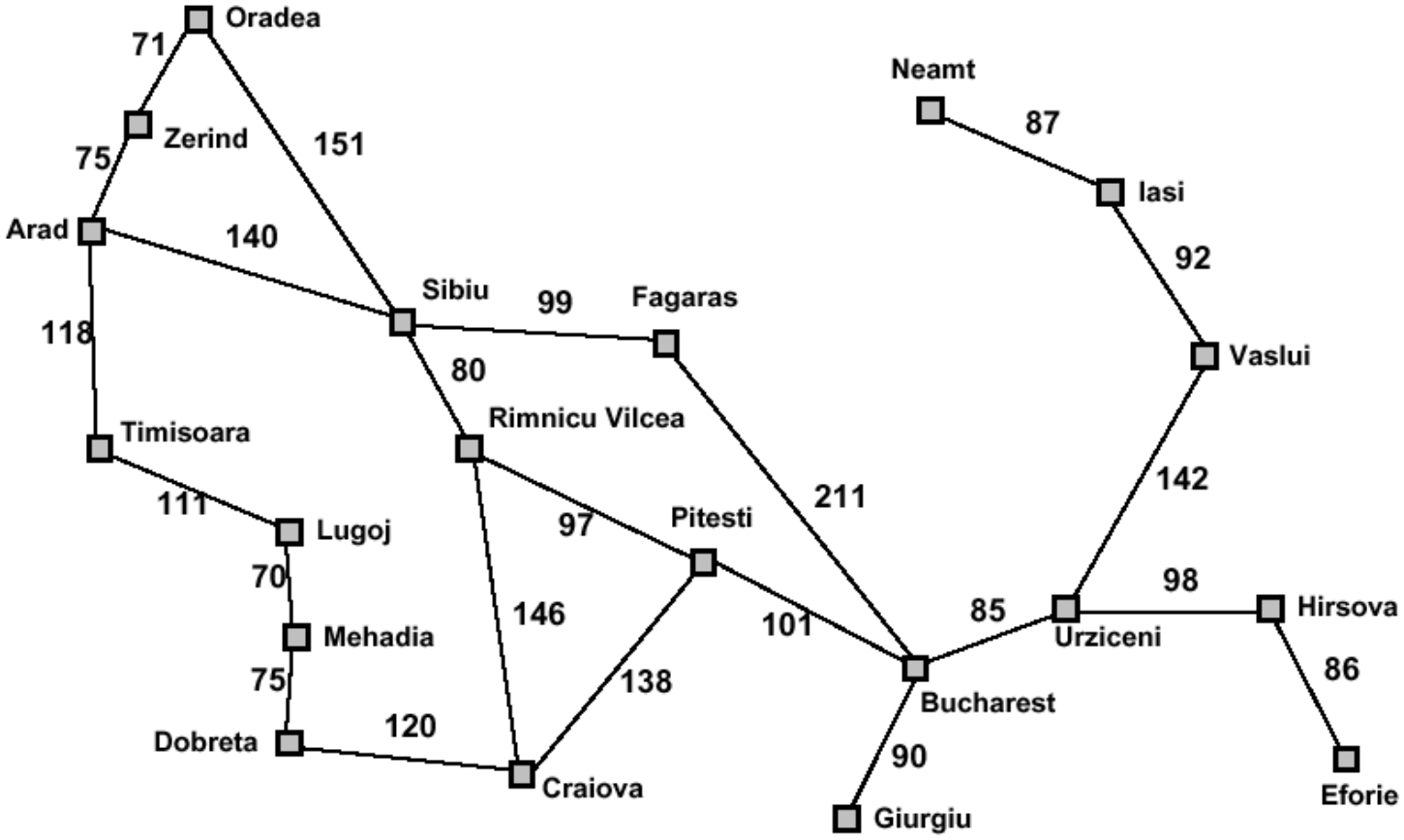
Tìm kiếm theo chiều sâu (Depth First Search) - Ý tưởng

- Tại mỗi bước, chọn trạng thái để phát triển là trạng thái được sinh ra sau cùng trong các trạng thái chờ phát triển
- Nói một cách khác, chúng ta sẽ quét cây tìm kiếm theo từng nhánh tính từ gốc trở đi
- Tương tự như thuật toán tìm kiếm chiều rộng chỉ khác là sử dụng một danh sách theo kiểu "ngăn xếp" (***stack LIFO***) chứ không phải theo kiểu "hàng đợi".

DFS – Ý tưởng

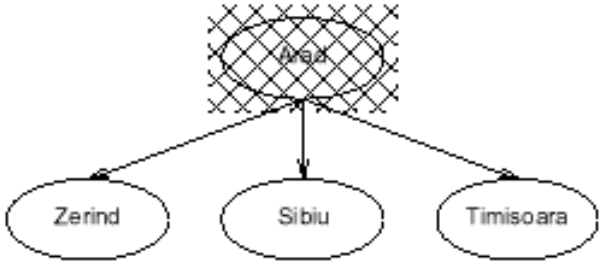


Ví dụ: Du lịch Romania với giá (cost) tính bằng km

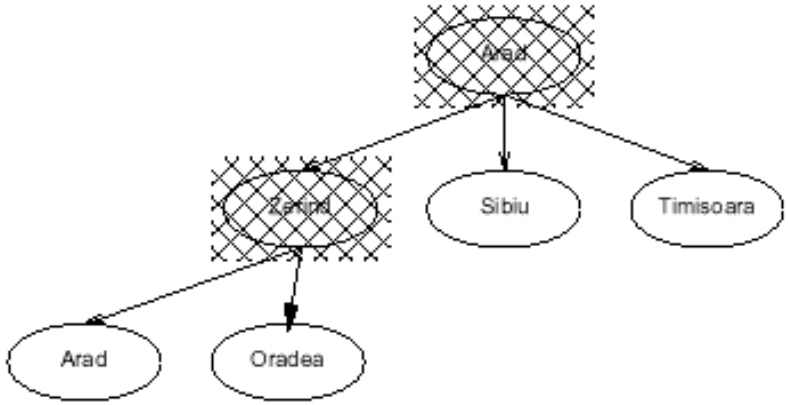


Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

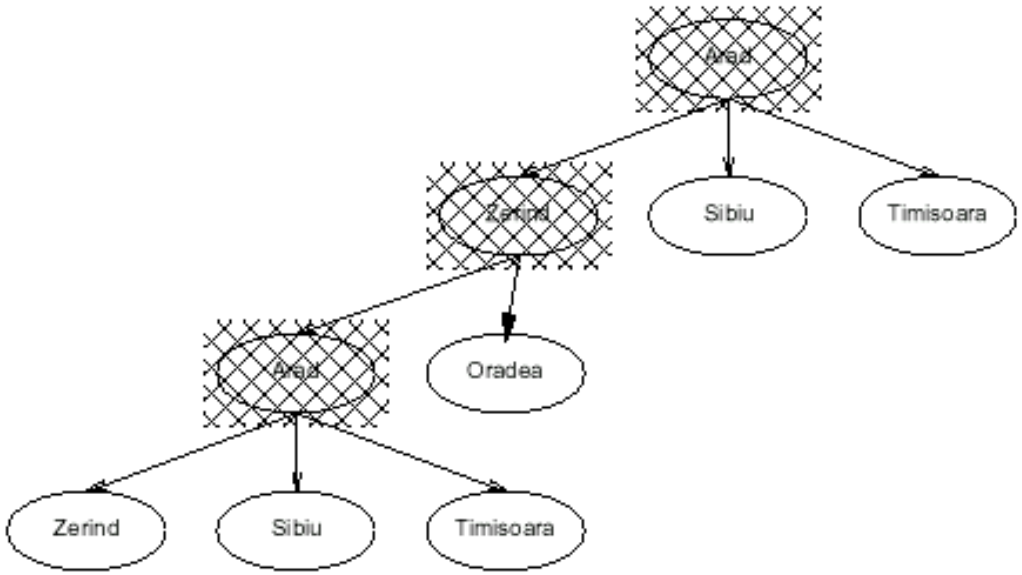
Ví dụ



Ví dụ



Ví dụ



I.e., depth-first search can perform infinite cyclic excursions
Need a finite, non-cyclic search space (or repeated-state checking)

Mô tả thuật toán DFS

Từ thuật toán tổng quát ta có:

1. Khởi tạo ngăn xếp L chỉ chứa trạng thái ban đầu
2. **Loop do**
   - 2.1 **If** (L rỗng) **then**
     {thông báo tìm kiếm thất bại; kết thúc}
   - 2.2 Loại trạng thái u ở đầu ngăn xếp L;
   - 2.3 **If** (u là trạng thái kết thúc) **then**
     {thông báo tìm kiếm thành công; kết thúc}
   - 2.4 **For** (mỗi trạng thái v kề u) **do**
     {Đặt v vào đầu ngăn xếp L;}

Tính chất của thuật toán DFS

- Completeness: No, fails in infinite state-space (yes if finite state space)
- Time complexity: $O(b^m)$
- Space complexity: $O(bm)$
- Optimality: No

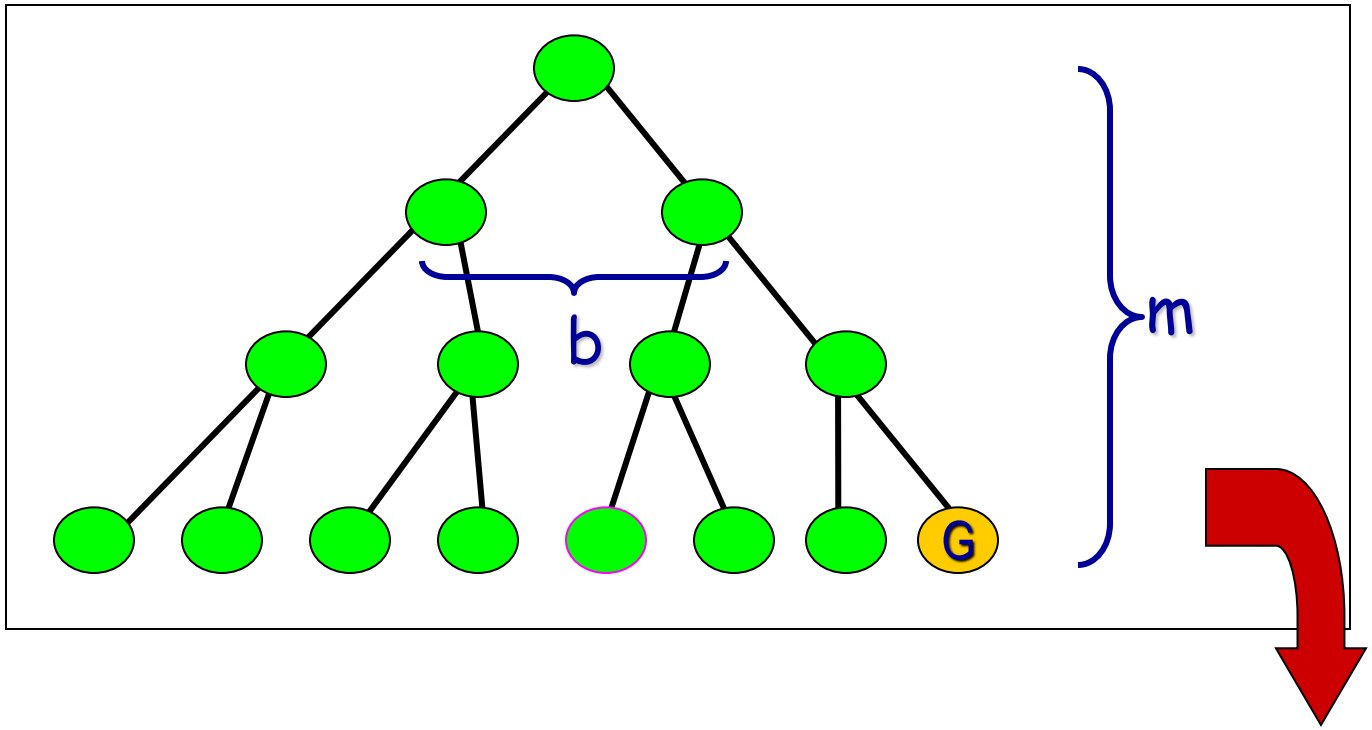
Remember:

b = branching factor

m = max depth of search tree

Độ phức tạp về thời gian

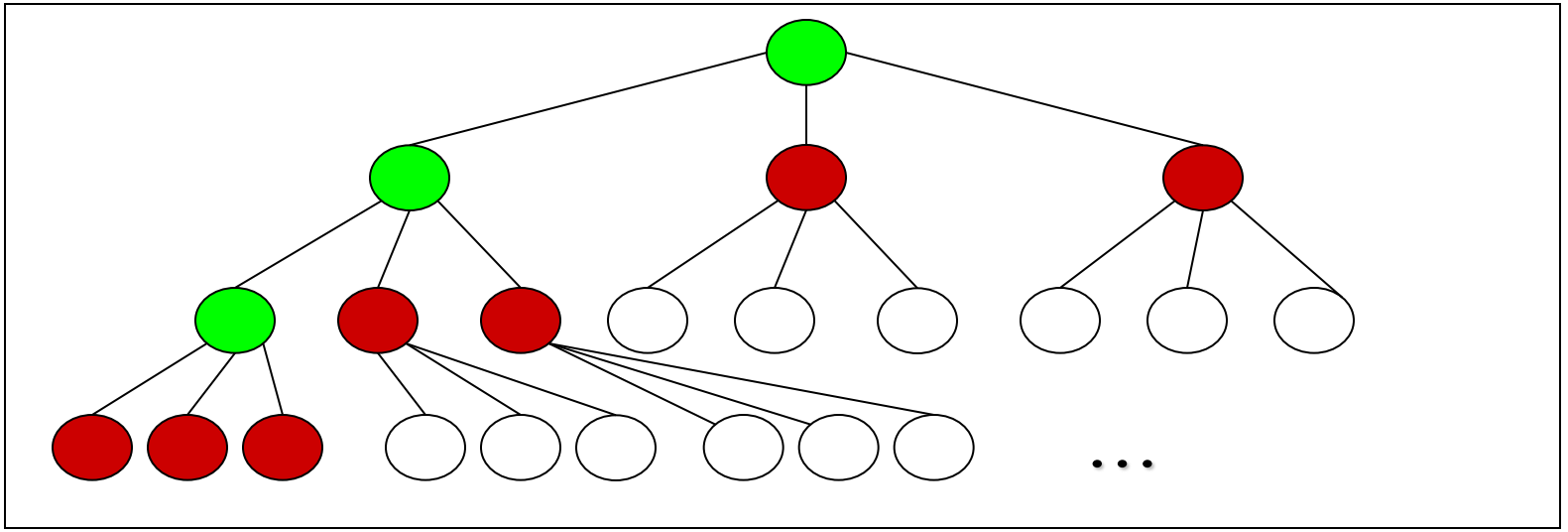
- In the worst case:
  - the (only) goal node may be on the right-most branch,



- Time complexity == $b^m + b^{m-1} + \ldots + 1 = \frac{b^{m+1} - 1}{b - 1}$
- Thus: $O(b^m)$

Độ phức tạp về không gian

- Largest number of nodes in QUEUE is reached in bottom left-most node.
- Example: m = 3, b = 3 :

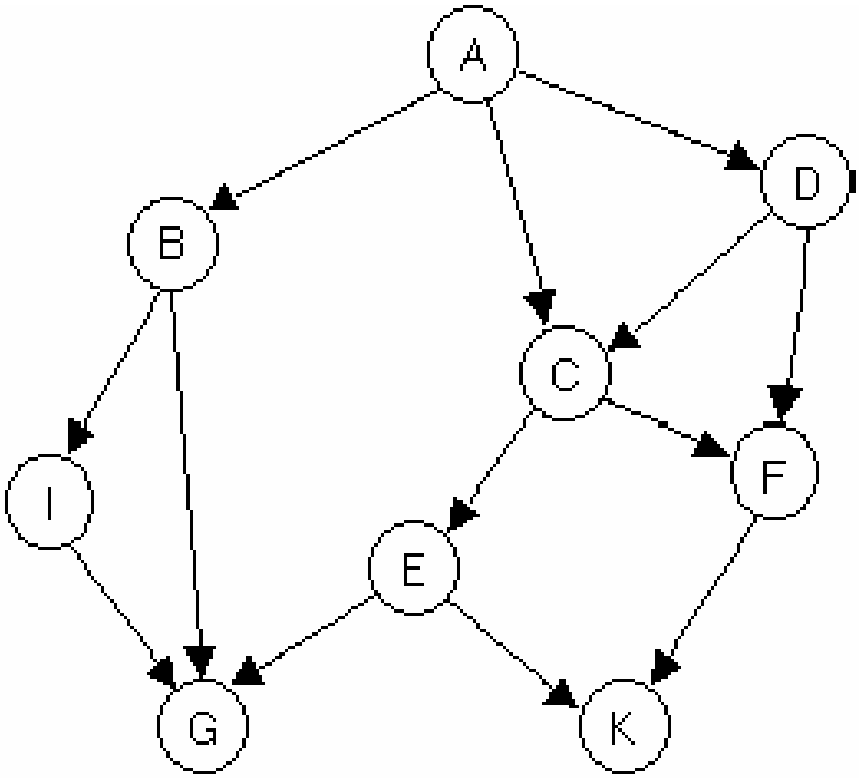


- QUEUE contains all  nodes. Thus: 7.
- In General: ((b-1) * m) + 1
- Order: O(m*b)

Tránh các trạng thái lặp (Avoiding repeated states)

- Trong cây tìm kiếm có thể chứa nhiều đỉnh ứng với cùng một trạng thái – các trạng thái này được gọi là trạng thái lặp
- Các thuật toán tìm kiếm sẽ lãng phí rất nhiều thời gian để phát triển lại các trạng thái mà ta đã gặp và đã phát triển
- Cách giải quyết: đánh dấu các trạng thái đã được sinh ra, không sinh ra lại những trạng thái đã được sinh ra

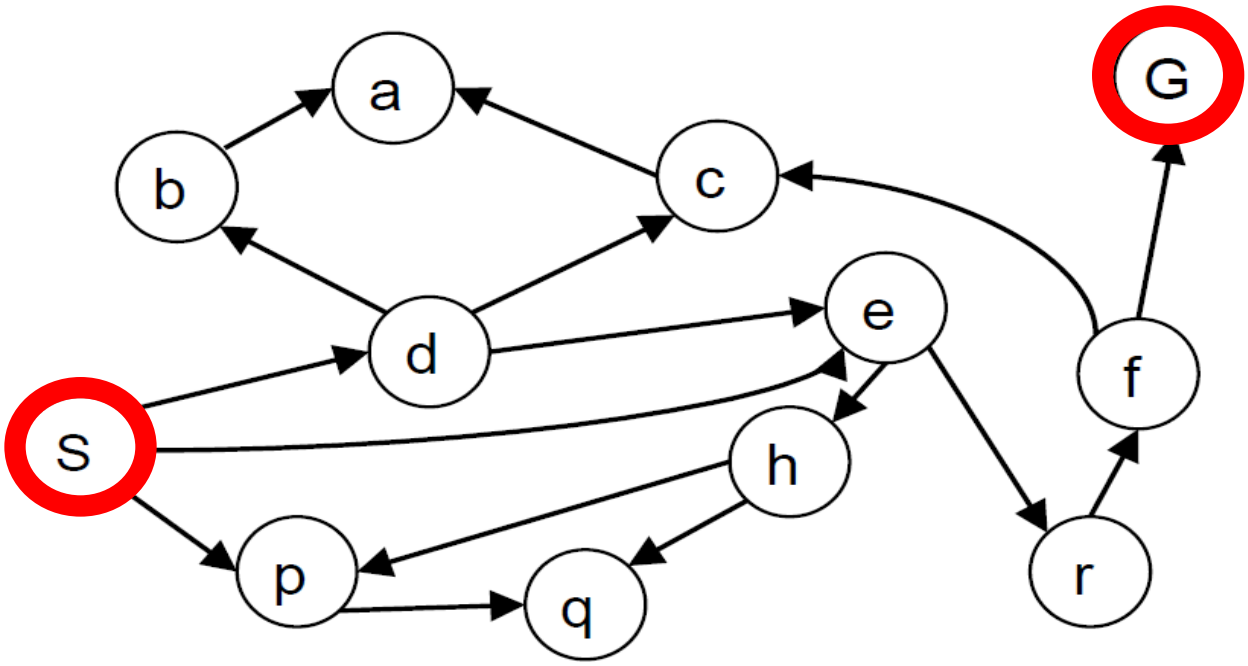
Bài tập (1)



Hãy tìm đường đi từ đỉnh A tới đỉnh K bằng :
-Tìm kiếm chiều rộng
-Tìm kiếm chiều sâu

Các bước	**Đỉnh u**	**Danh sách L (FIFO/LIFO)**
1	-	{A}

Bài tập (2)



Hãy tìm đường đi từ đỉnh S tới đỉnh G bằng :

-Tìm kiếm chiều rộng

-Tìm kiếm chiều sâu

Tìm kiếm theo chiều sâu có giới hạn (Depth-limited search)

Is a depth-first search with depth limit l

Implementation:
Nodes at depth l have no successors.

Complete: if cutoff chosen appropriately then it is guaranteed to find a solution.

Optimal: it does not guarantee to find the least-cost solution

Tìm kiếm theo chều sâu lặp (Iterative deepening search)

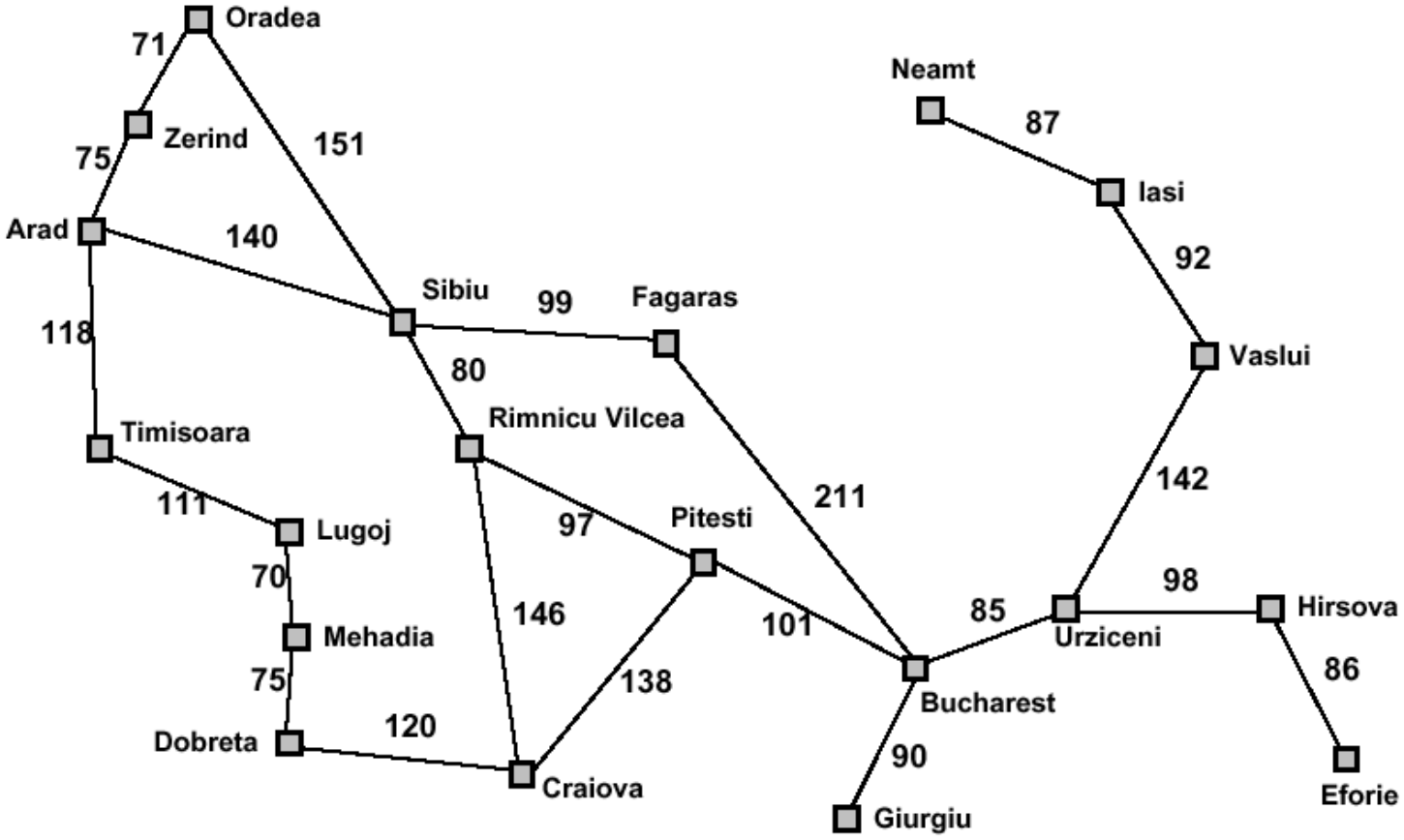
Function Iterative-deepening-Search(*problem*) **returns** a solution, or failure

```
for depth = 0 to ∞ do
    result ← Depth-Limited-Search(problem, depth)
    if result succeeds then return result
end
return failure
```

Combines the best of breadth-first and depth-first search strategies.

- Completeness: Yes,
- Time complexity: $O(b^d)$
- Space complexity: $O(bd)$
- Optimality: Yes, if step cost = 1

Quay lại ví dụ du lịch Romania



Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Adobe Acrobat - [chapter03.pdf]
File Edit Document Tools View Window Help

Iterative deepening search $l = 0$



153% 53 of 63 11 x 8.5 in
Start CS 564 laurent talks CS 561a Aima Adobe Acr... Microsoft Pow... Telnet - pollux... books Adobe Photos... 8:22 PM

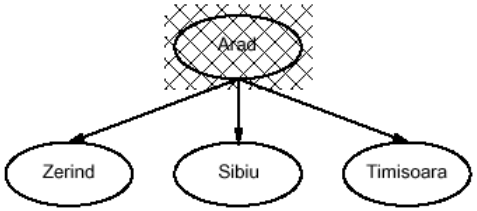
Adobe Acrobat - [chapter03.pdf]
File Edit Document Tools View Window Help

Iterative deepening search $l = 1$



153% 54 of 63 11 x 8.5 in
Start CS 564 laurent talks CS 561a Aima Adobe Acr... Microsoft Pow... Telnet - pollux... books Adobe Photos... 8:24 PM

Adobe Acrobat - [chapter03.pdf]
File Edit Document Tools View Window Help



153% 55 of 63 11 x 8.5 in
Start CS 564 laurent talks CS 561a Aima Adobe Acr... Microsoft Pow... Telnet - pollux... books Adobe Photos... 8:24 PM

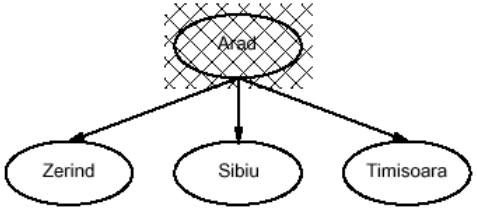
Adobe Acrobat - [chapter03.pdf]
File Edit Document Tools View Window Help

Iterative deepening search $l = 2$



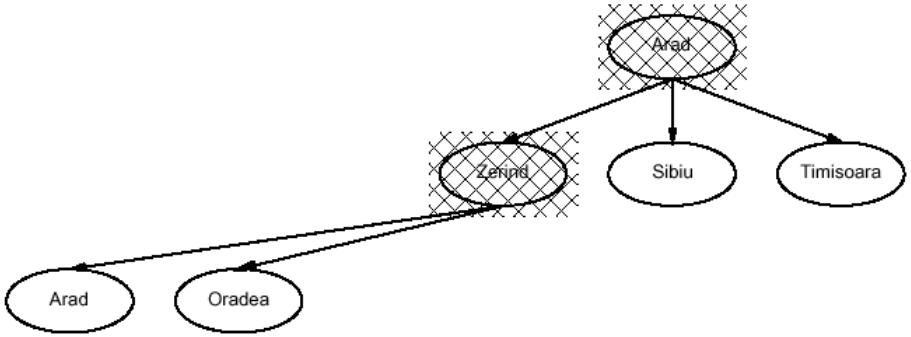
153% 56 of 63 11 x 8.5 in
Start CS 564 laurent talks CS 561a Aima Adobe Acr... Microsoft Pow... Telnet - pollux... books Adobe Photos... 8:25 PM

Adobe Acrobat - [chapter03.pdf]
File Edit Document Tools View Window Help



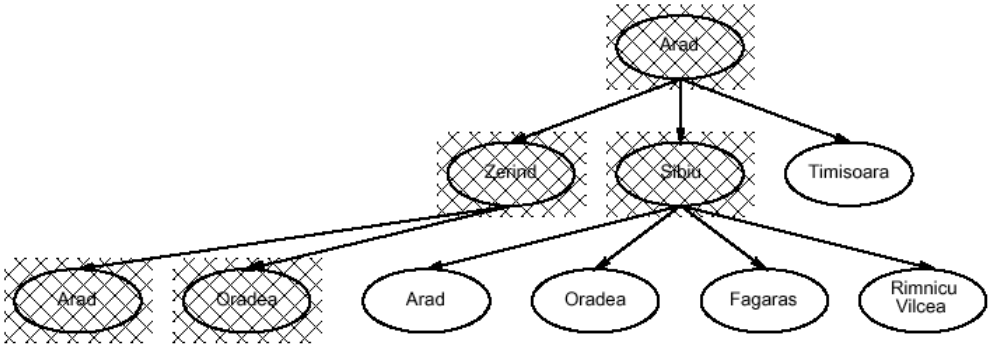
153% 57 of 63 11 x 8.5 in
Start CS 564 laurent talks CS 561a Aima Adobe Acr... Microsoft Pow... Telnet - pollux... books Adobe Photos... 8:25 PM

Adobe Acrobat - [chapter03.pdf]
File Edit Document Tools View Window Help

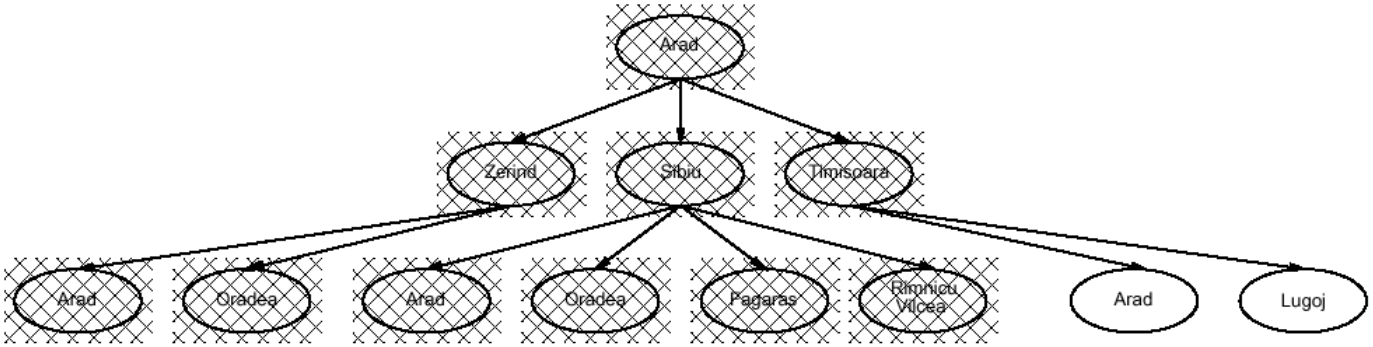


153% 58 of 63 11 x 8.5 in
Start CS 564 laurent talks CS 561a Aima Adobe Acr... Microsoft Pow... Telnet - pollux... books Adobe Photos... 8:26 PM

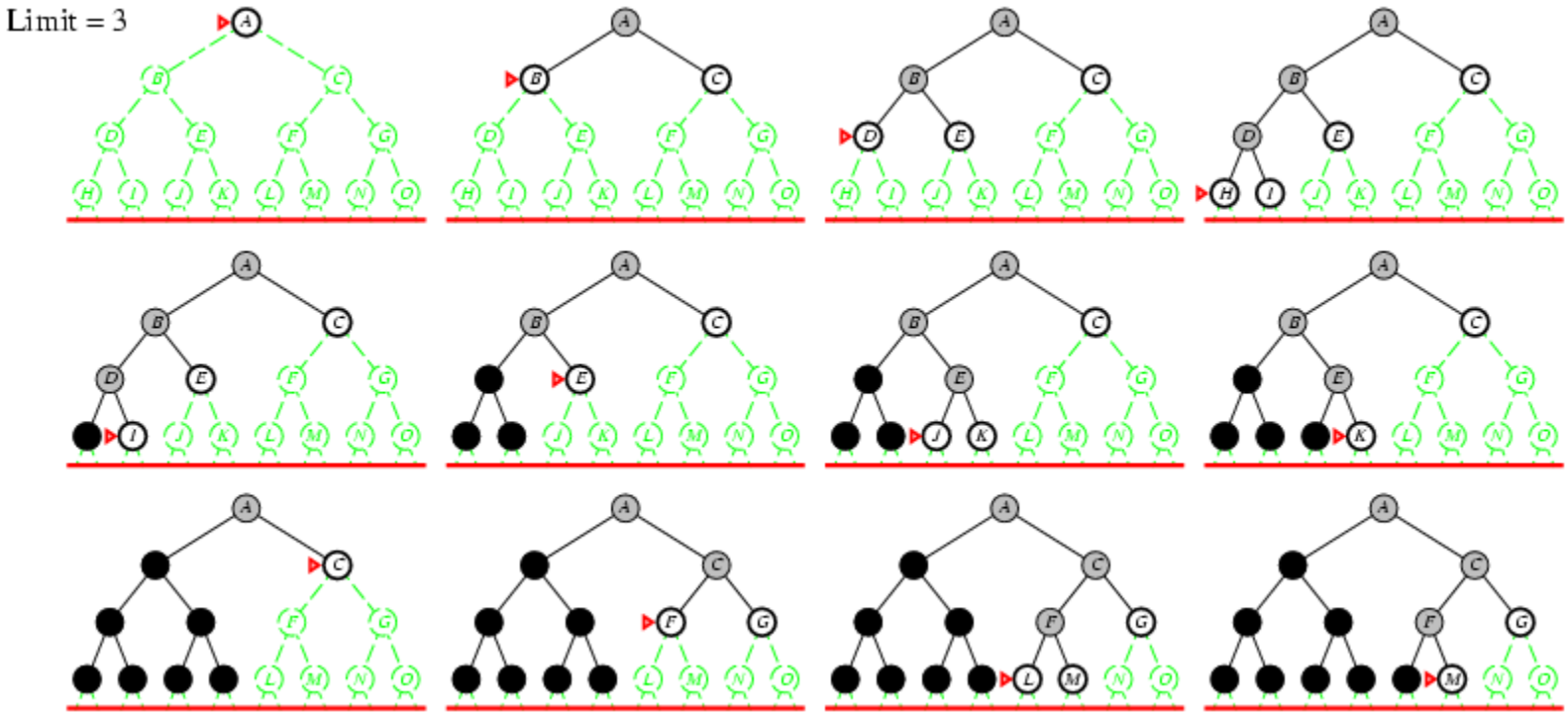
Adobe Acrobat - [chapter03.pdf]
File Edit Document Tools View Window Help



153% 59 of 63 11 x 8.5 in
Start CS 564 laurent talks CS 561a Aima Adobe Acr... Microsoft Pow... Telnet - pollux... books Adobe Photos... 8:26 PM


Arad
Zerind
Sibiu
Timisoara
Arad
Oradea
Arad
Oradea
Pagaras
Rimnicu Vilcea
Arad
Lugoj

Ví dụ

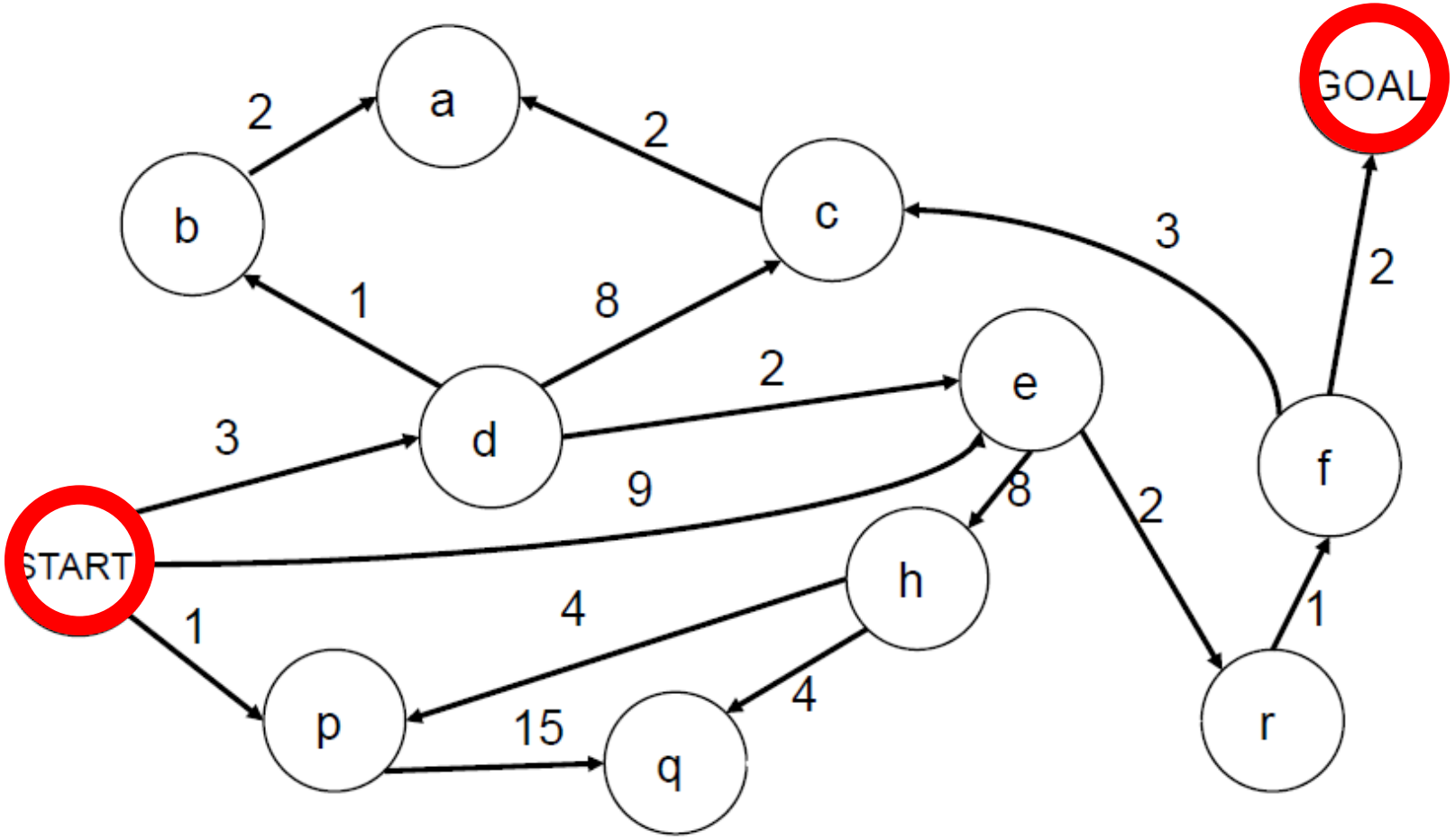


In general, iterative deepening is preferred to depth-first or breadth-first when search space large and depth of solution not known.

Tìm kiếm chi phí đều/ cực tiểu (Uniform-cost search)

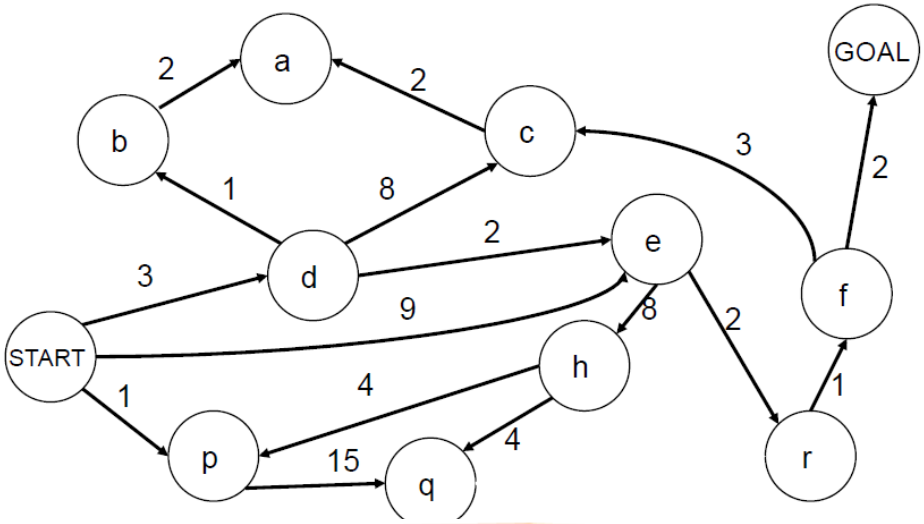
- For each frontier node (queue), save the total cost of the path from the initial state to that node
- Expand the frontier node with the lowest path cost
- Implementation: *frontier* is a priority queue ordered by path cost
- Equivalent to breadth-first if step costs all equal
- Equivalent to Dijkstra's algorithm in general

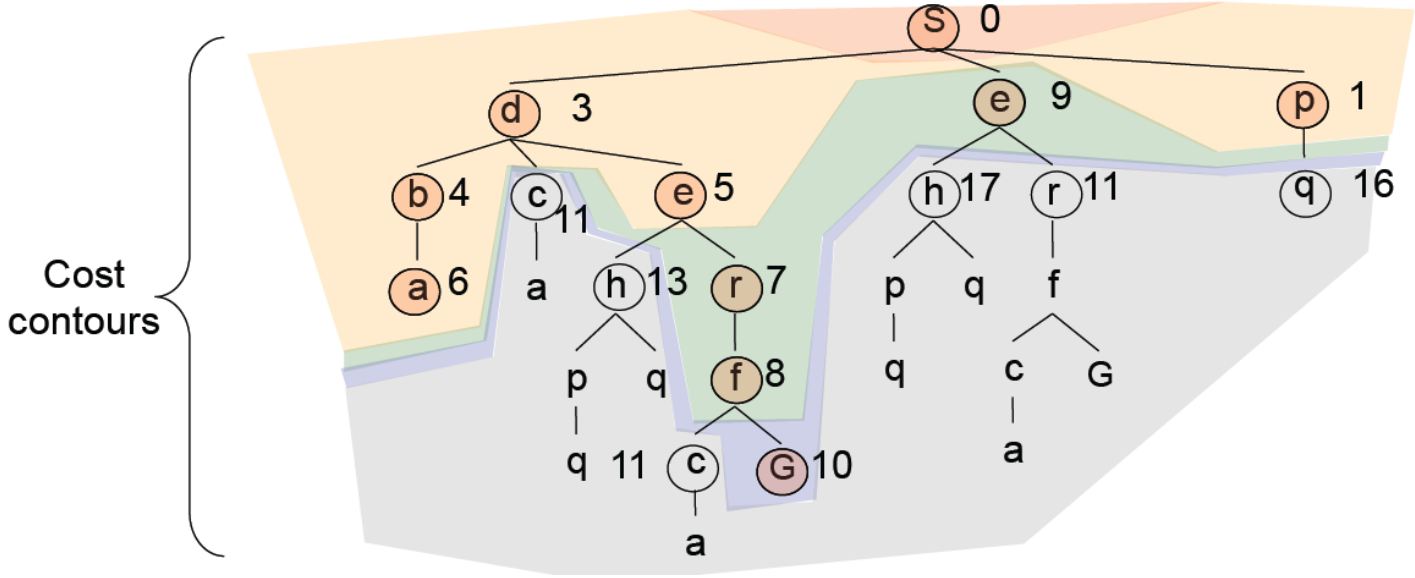
Ví dụ



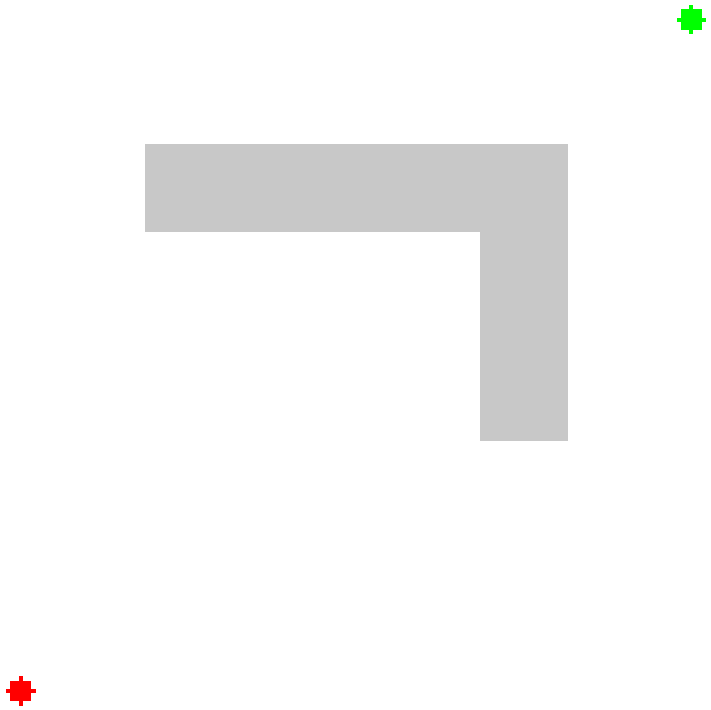
Ví dụ

- Expansion order:
(S,p,d,b,e,a,r,f,e,G)





Một ví dụ khác



Source: Wikipedia

Tính chất của tìm kiếm chi phí đều

- **Complete?**

  Yes, if step cost is greater than some positive constant ε (we don't want infinite sequences of steps that have a finite total cost)

- **Optimal?**

  Yes

Độ tối ưu của tìm kiếm chi phí đều

- **Graph separation property**: every path from the initial state to an unexplored state has to pass through a state on the frontier
  - Proved inductively

- Optimality of UCS: proof by contradiction
  - Suppose UCS terminates at goal state n with path cost $g(n)$ but there exists another goal state n' with $g(n') < g(n)$
  - By the graph separation property, there must exist a node n'' on the frontier that is on the optimal path to n'
  - But because $g(n'') \leq g(n') < g(n)$, n'' should have been expanded first!

Tính chất của tìm kiếm chi phí đều

- **Complete?**

  Yes, if step cost is greater than some positive constant ε (we don't want infinite sequences of steps that have a finite total cost)

- **Optimal?**

  Yes – nodes expanded in increasing order of path cost

- **Time?**

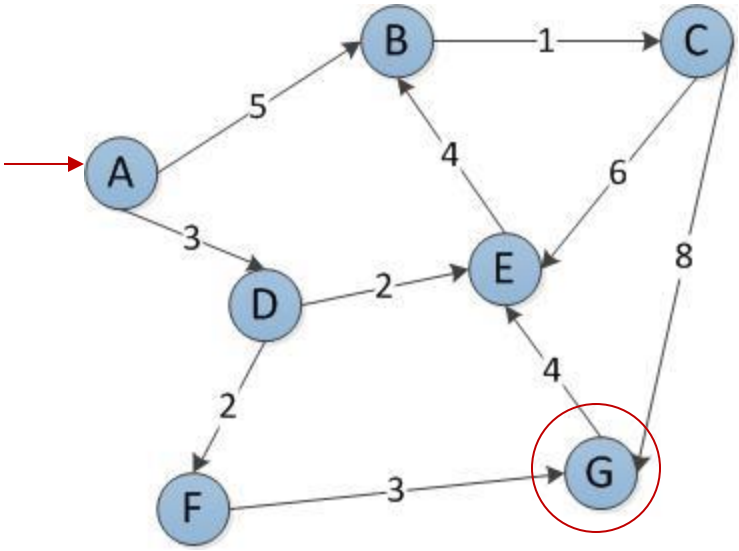
  Number of nodes with path cost ≤ cost of optimal solution (C^*), $O(b^{C^*/\varepsilon})$

  This can be greater than $O(b^d)$: the search can explore long paths consisting of small steps before exploring shorter paths consisting of larger steps

- **Space?**

  $O(b^{C^*/\varepsilon})$

Bài tập 3 – Tìm kiếm chi phí đều



Các bước	Biên giới: tập các đối tượng (nút và chi phí)	Mở rộng	Đã xét: tập các nút đã xét
1	{(A,0)}	A	∅
?	?	?	?

So sánh các thuật toán tìm kiếm mù

Criterion	Breadth first	Uniform cost	Depth-first	Depth-limited	Iterative deepening
Time	b^d	b^d	b^m	b^l	b^d
Space	b^d	b^d	bm	bl	bd
Optimal?	Yes	Yes	No	No	Yes
Complete?	Yes	Yes	No	Yes, if l≥d	Yes

- *b* – max branching factor of the search tree
- *d* – depth of the least-cost solution
- *m* – max depth of the state-space (may be infinity)
- *l* – depth cutoff

Kết luận

- Problem formulation usually requires abstracting away real-world details to define a state space that can be explored using computer algorithms.

- Once problem is formulated in abstract form, complexity analysis helps us picking out best algorithm to solve problem.

- Variety of uninformed search strategies; difference lies in method used to pick node that will be further expanded.